



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

Факультет «Информатика и вычислительная техника»
Кафедра «Кибербезопасность информационных систем»

Зав. кафедрой _____ «КБИС»
(подпись) А. И. Жуков
«24» 12 _____ (И.О.Ф.)
2018 г.

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к дипломному проекту (работе) на тему:

ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КРИПТОГРАФИЧЕСКОЙ СИСТЕМЫ
NTRUENCRYPT

Автор дипломного проекта (работы) _____

(подпись, дата)

А. В. Селёва

(И.О.Ф.)

Обозначение дипломного проекта 10.05.01.480000.000 ДП Группа ВКБ61

Специальность 10.05.01
(код)

Компьютерная безопасность
(наименование специальности)

Руководитель проекта (работы) _____

(подпись, дата)

д.ф.-м.н., профессор Черкесова Л.В.
(должность, И.О.Ф.)

Нормоконтроль _____

(подпись, дата)

Зав. каф. ИСУКОВ А.И.
(должность, И.О.Ф.)

Ростов-на-Дону
2018



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

Факультет «Информатика и вычислительная техника»
Кафедра «Кибербезопасность информационных систем»

Зав. кафедрой

«КБИС»

А. И. Жуков
(И.О.Ф.)

«11»

2018 г.

ЗАДАНИЕ

на дипломный проект (работу)

Обучающийся Селёва А. В. Группа ВКБ61

Обозначение дипломного проекта (работы) 10.05.01.480000.000 ДП

Тема ПРОГРАММНАЯ РЕАЛИЗАЦИЯ КРИПТОГРАФИЧЕСКОЙ СИСТЕМЫ
NTRUEncrypt

Утверждена приказом по ДГТУ № 3672-ЛС-О от «21» сентября 2018 г.

Срок представления проекта (работы) к защите «11» декабря 2018 г.

Исходные данные для дипломного проекта (работы)

Справочная литература по криптографии, научные статьи и диссертации по материалам криптографической системы NTRUEncrypt, справочная литература для среды программной разработки, электронные ресурсы статистических данных.

Содержание пояснительной записки

ВВЕДЕНИЕ:

Актуальность работы, цель исследования, задачи исследования, объект исследования, предмет исследования, научная новизна.

Наименование и содержание разделов

1 Безопасность информационных систем.

Рассмотрены и кратко описаны основные положения безопасности информационных систем, источники угроз ИБ и средства защиты информации. Дано описание задачи теории решеток, на основе которой разработан алгоритм NTRUEncrypt.

2 Алгоритмизация программных средств.

Приведены подробные алгоритмы как самой программы NTRUEncrypt, так и ее модификации. Описаны доказательства работы этих двух алгоритмов, а также перечислен ряд преимуществ модификации по сравнению с исходной программой. Приведены графические данные, наглядно демонстрирующие преимущества криптосистемы NTRUEncrypt.

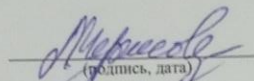
3 Программное конструирование системы NTRUEncrypt.

Разработка программных продуктов. Описание выбранного языка программирования, общей структуры программ и приведена их обобщенная схема работы. Приведены рисунки, демонстрирующие работу каждого режима работы программ и каждого модуля с дополнительными пояснениями автора. Проведены эксперименты. Зафиксированы результаты, построены графики.

ЗАКЛЮЧЕНИЕ

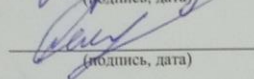
Основные выводы о проделанной работе, оценка достижения цели.

Руководитель проекта (работы)


(подпись, дата)

д.ф-м.н., профессор Черкесова Л.В.
(должность, И.О.Ф.)

Задание принял к исполнению


(подпись, дата)

А. В. Селёва
(И.О.Ф.)

сис
рес
кри
отв
зал
поз
прс
пре

- 5,

NTE
and
degr
of t
cryp
reso
spee

5, so

АННОТАЦИЯ

Данная работа посвящена программной реализации криптографической системы NTRUEncrypt и её модификации, предоставляющие эффективный ресурс для шифрования и дешифрации файлов данных. Разработанная автором криптосистема обладает определенной степенью криптостойкости по отношению к возможным квантовым атакам за счет сложности решения заложенной в алгоритм задачи теории решеток. Модификация криптосистемы позволяет улучшить показатели быстродействия и потребления ресурсов программного обеспечения. Проведены экспериментальные доказательства преимуществ модификации по характеристикам скорости работы алгоритма.

Объем работы – 99, количество иллюстраций – 36, таблиц – 5, приложений – 5, использованных источников – 23.

ABSTRACT

This work is devoted to the software implementation of the cryptographic system NTRUEncrypt and its modifications, providing an effective resource for encryption and decryption of data files. The author developed the cryptosystem has a certain degree of reliability in relation to the possible quantum attacks due to the complexity of the decisions inherent in the algorithm theory of lattices. Modification of the cryptosystem allows to improve the performance and consumption of software resources. Experimental evidence of benefits of modifying the characteristics of the speed of the algorithm.

The volume of work – 99, the number of illustrations-36, tables-5, applications-5, sources-23.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	7
1 Безопасность информационных систем	11
1.1 Источники угроз ИБ	11
1.2 Средства защиты информации	13
1.3 Криптографическая защита информации	15
1.4 Алгоритмы ассиметричного шифрования	16
1.5 Асимметричная криптосистема RSA	19
1.6 Введение в теорию квантовых компьютеров	20
1.6.1 Теория квантовой информации	20
1.6.2 Алгоритм Питера Шора	21
1.6.3 Квантовые гейты	23
1.6.4 Отличие кубитов от классических битов	23
1.6.5 Квантовые вычисления	25
1.6.6 Квантовые алгоритмы	26
1.6.7 Применение квантовых гейтов	29
1.7 Кратчайший вектор решетки	30
1.8 Выводы к главе 1	37
2 Алгоритмизация программных средств	38
2.1 Система шифрования NTRUEncrypt	38
2.1.1 Алгоритм генерации ключей системы	39
2.1.2 Алгоритм шифрования сообщения	40
2.1.3 Алгоритм расшифрование шифротекста	40
2.1.4 Доказательство работы алгоритма	41

					10.05.01.480000.000 ПЗ		
Изм.	Лист	№ докум.	Подпись	Дата			
Разраб.		Селёва А. В.			Программная реализация криптографической системы NTRUEncrypt	Лит.	Лист
Провер.		Черкесова Л. В.					5
							99
Н. Контр.		Жуков А. И.				ДГТУ кафедра КБИС	
Утверд.		Жуков А. И.					

2.2 Модификация алгоритма NTRUEncrypt	42
2.2.1 Доказательство работы модифицированного алгоритма	44
2.3 Преимущества системы NTRUEncrypt	45
2.4 Выводы к главе 2	46
3 Программное конструирование системы NTRUEncrypt	47
3.1 Выбор языка программирования	47
3.2 Структура разработанных программ	47
3.3 Программная реализация алгоритма NTRUEncrypt	50
3.3.1 Демонстрационный режим работы программы NTRUEncrypt	51
3.3.2 Режим работы с файлом программы NTRUEncrypt	53
3.4 Программная реализация модифицированного алгоритма	56
3.4.1 Демонстрационный режим работы модифицированной программы	57
3.4.2 Режим работы с файлом модифицированной программы	58
3.5 Экспериментальное исследование	59
3.5.1 Результаты экспериментального исследования	60
3.6 Выводы к главе 3	66
ЗАКЛЮЧЕНИЕ	67
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	69
ПРИЛОЖЕНИЕ А Техническое задание	72
ПРИЛОЖЕНИЕ Б Руководство системного программиста	77
ПРИЛОЖЕНИЕ В Руководство программиста	80
ПРИЛОЖЕНИЕ Г Руководство оператора	82
ПРИЛОЖЕНИЕ Д Исходный код программного комплекса	84

ВВЕДЕНИЕ

Авторство знаменитой фразы о том, что кто владеет информацией, тот владеет миром приписывают и философу-математику Фрэнсису Бэкону (1561 г.р.) и банкиру-спекулянту Натану Ротшильду (1777 г.р.), а также великому Британскому политику У. Черчиллю (1874 г.р.). Нам не так важно кто первый осознал этот факт – а то, что на протяжении вот уже более 400-х лет владеющий необходимой информацией продолжает владеть умами людей, может просчитывать ход событий, определять политику государств во всем мире. Со времен Ф. Бэкона только сильно изменились способы обмена информацией: Н. Ротшильд уже использовал голубиную почту, а во времена Черчилля были такие средства как радио и телеграф, которые могли значительно увеличить объемы и скорость передаваемой информации.

Вплоть до 20-го века для обмена информацией люди в основном использовали устную речь и обменивались письмами, и уже тогда, особо продвинутые и коварные, изобрели язык символов и жестов, понятный только посвященным, и придумали тайнопись и примитивное шифрование текстов.

Первые шифровальные машины изобрел Э.Геберн в 1917г., но затем, в связи с бурным развитием средств связи, на основе криптологии начали появляться более совершенные шифраторы, в основном, для нужд военных ведомств. Пожалуй, самая известная из этой серии — нацистская «Энигма». Англичане сумели взломать код «Энигмы» и, как считают некоторые историки, именно это событие было решающим фактором в победе союзников во Второй Мировой войне. Вот такую цену заплатил Гитлеровский режим за потерю военной тайны.

Развитие прогрессивных технологий подарило человечеству великое множество достижений, но в то же время породило и массу проблем. Человеческий гений, сталкиваясь с одними проблемами и успешно их разрешая, непременно наталкивается на новые, и этот процесс бесконечен

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		7

согласно законам развития. Сейчас мы имеем такие цифровые технологии, о которых даже не смели мечтать всего 20-30 лет назад. Не зря говорят, что новое – это хорошо забытое старое, но одна проблема остается вечной на протяжении многих веков, это проблема защиты информации. Человечество по-разному решало эту проблему в зависимости от эпохи и уровня развития производственных сил в данное время. Переосмысление кибернетики, изобретение компьютера и дальнейшее бурное развитие информационных технологий с 60-х годов прошлого века, сделали проблему защиты информации одной из самых важных при планировании внедрения цифровых сетей и является актуальной по сей день.

Понимают это и государственные чиновники. Наша страна, как и все так называемые цивилизованные страны, также использует всю свою законодательную и карательную мощь для защиты авторских прав и персональных данных своих граждан. Федеральным Законом Российской Федерации «Об информации, информатизации и защите информации» четко определено, что всевозможные документы и другие информационные ресурсы, в том числе и находящиеся в информационных системах, подлежат защите наравне с материальным имуществом собственника. Он также напрямую относит к категории конфиденциальной информации любую информацию о гражданах (персональные данные). ФЗ РФ «О банках и банковской деятельности» стоит на страже тайны вкладов и ограничивает доступ к операциям и счетам клиентов банков. Нормы в отношении служебной и коммерческой тайны определяются Гражданским кодексом РФ (ст.139).

Информация сегодня стоит очень дорого и тот, кто не научится её охранять, рискует очень многим. В США, например, потери от компьютерной преступности в 2017 году [1] некоторые аналитики оценивают более чем на 120 млрд долларов, в странах Евросоюза — на 32 млрд. евро. В среднем ущерб от одного компьютерного преступления составляет от 460 тыс. и до 1,1 млрд евро. В США некоторые фирмы теряли до 5,5млр долларов. Если верить

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		8

статистики, то около 80% компаний и фирм, разных форм собственности и веса в бизнесе, несут убытки из-за недостаточно надежного обеспечения безопасности данных.

Сегодня, наверное, никто не сможет с уверенностью назвать точную цифру суммарных потерь от компьютерных преступлений, связанных с несанкционированным доступом к информации. Это объясняется, прежде всего, нежеланием пострадавших компаний обнародовать информацию о своих потерях, а также тем, что не всегда потери от хищения информации можно точно оценить в денежном эквиваленте.

Криптография неразрывно связана с передачей данных, и помимо обеспечения авторизации пользователей, она гарантирует целостность передаваемой информации и конфиденциальность. Очевидно, что для таких организаций как банки, военные ведомства, государственные учреждения применение криптостойких систем просто необходимо, но на самом деле, мы используем криптографию ежедневно. Она распространила свое влияние на многие области наших рутинных действий, начиная от использования телефона для снятия наличных в банкомате, заканчивая просмотром DVD-фильма или открытием автомобиля с пульта управления.

Толчком для разработки новой криптографической системы послужила статья [2], в которой описывались возможности квантовых компьютеров, и какая от них исходит угроза взлома для всех широко используемых в настоящее время криптографических алгоритмов.

Актуальность работы заключается в том, что представленная программная разработка обладает криптостойкостью по отношению к возможным квантовым атакам и, согласно статистике, превосходит своих аналогов (например, криптосистему RSA) по характеристикам скорости работы алгоритма и по количеству затрачиваемых ресурсов памяти.

Объектом исследования является криптосистема NTRUEncrypt.

					10.05.01.480000.000 ПЗ	Лист
						9
Изм.	Лист	№ докум.	Подпись	Дата		

Предмет исследования: алгоритмическая сложность разработки криптосистемы NTRUEncrypt и ее модификации.

Целью настоящего исследования является разработка эффективного постквантового криптографического алгоритма NTRUEncrypt, обладающего высокой криптостойкостью по отношению к атакам при помощи квантового компьютера, а также разработка модификации предложенного алгоритма, анализ и экспериментальное доказательство преимуществ последнего.

В соответствии с поставленной целью были определены задачи:

1. Исследовать алгоритм работы криптосистемы NTRUEncrypt. Разработать алгоритм модификации NTRUEncrypt.
3. Реализовать программные средства криптосистемы NTRUEncrypt и ее модификации.
4. Провести и проанализировать сравнение двух программ между собой и сравнить их с существующим аналогом – криптосистемой RSA.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		10

1 Безопасность информационных систем

Информация сегодня стоит очень дорого и тот, кто не научится её охранять, тот рискует очень многим. Практически все компании видят информацию, как источник ценных ресурсов, стараются её защитить и поэтому задача обеспечения конфиденциальности информации становится на первый план. Рассмотрим понятие информационной безопасности (ИБ) системы более подробно.

ИБ системы – это свойство системы, которое гарантирует корректное функционирование её процессов, то есть обеспечивает целостность и конфиденциальность данных. Чтобы секретность информации не ставилась под сомнения необходимо внедрение дополнительной защиты от случайного уничтожения информации или от возможного доступа к секретной информации лица, не обладающего правами доступа.

Обеспечение целостности информации означает отсутствие возможности изменения или уничтожения информации как случайным образом, так и путем несанкционированного вмешательства. Конфиденциальность информации означает отсутствие возможности перехвата передаваемой или хранящейся на устройстве информации любым несанкционированным способом.

1.1 Источники угроз ИБ

Приведем пример некоторых источников угроз ИБ:

- антропогенные. Возникают в результате случайных или преднамеренных действий пользователя;
- техногенные. Возникают в результате использования старых программных средств или в результате ошибок в программном обеспечении;
- стихийные. Возникают в результате природных катастроф.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		11

Антропогенные источники можно разделить на следующие категории:

- на внешние и внутренние источники. В роли внешних источников выступают несанкционированные действия третьих лиц, в роли внутренних источников выступают действия штатных сотрудников;
- на случайные и преднамеренные действия пользователей.

На сегодняшний день существует довольно большое множество различных способов утечки информации и незаконных путей доступа к ней.

Приведем примеры некоторые из них:

- перехват данных при их передаче по сети;
- изменение данных (сообщение модифицируется или заменяется другим и отправляется адресату);
- замена авторства сообщения (злоумышленник может заменить авторство своего сообщения и отправить его от имени пользователя);
- злоумышленником могут быть использованы различного рода недостатки операционной системы или установленных ПО;
- получение копий информационных носителей и файлов;
- несанкционированный доступ к аппаратуре и линиям связи;
- присвоение злоумышленником прав зарегистрированного пользователя;
- использование вирусов в компьютерных системах.

Чтобы обеспечить необходимый уровень безопасности систем данных необходимо использовать комплекс организационно-технических мер и программно-аппаратных средств нацеленных на предотвращение угроз безопасности.

Стоит отметить, что не существует полностью безопасной системы защиты данных. Зачастую перед руководителем стоит задача выбора между качеством работы и уровнем защиты.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		12

1.2 Средства защиты информации

Средства защиты информации (СЗИ) делятся на:

- СЗИ от незаконного доступа к данным;
- СЗИ в сетях передачи данных;
- защита данных криптографическими методами;
- использование ЭЦП (электронной цифровой подписи);
- СЗИ от вирусов системы.

СЗИ от незаконного доступа к данным.

Подразумевает прохождение пользователем этапов идентификации, аутентификации и авторизации для получения прав доступа к ресурсам системы данных.

Идентификация – это процедура определения пользователя по его уникальному идентификатору.

Аутентификация – это процедура проверки подлинности. Пользователь предоставляет свой уникальный идентификатор, а система проверяет действительно ли это тот человек, за которого он себя выдает. Чаще всего для целей аутентификации используют пароль, который закрепляется за пользователем и хранится в системе.

Авторизация – предоставление прав доступа пользователю к конкретному ресурсу системы. Авторизация выполняет функцию по разграничению прав доступа к ресурсам.

СЗИ в сетях передачи данных.

Во многих организациях локальная сеть имеет выход в Интернет и для их защиты зачастую используют так называемые брандмауэры(firewalls) или межсетевые экраны.

Брандмауэр (firewall) – это программное или аппаратное средство, которое разделяет сеть на локальную и Интернет, при этом формируется набор

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		13

правил которые задают условия прохождение данных из одной части сети в другую.

Электронная цифровая подпись (ЭЦП).

При передаче по сети исходное сообщение может быть модифицировано или заменено на другое сообщение злоумышленником. Для предотвращения такой возможности необходимо отправлять исходное сообщение вместе с его ЭЦП.

Электронная цифровая подпись получается из исходного сообщения путем применения к нему определенного криптографического алгоритма с использованием секретного ключа, который известен только владельцу сообщения. ЭЦП представляет собой последовательность символов и при наличии открытого ключа дает информацию о том было ли изменено сообщение при передаче и действительно ли принадлежит его автору.

Другими словами, сформированная ЭЦП представляет собой зашифрованное при помощи закрытого ключа исходное сообщение. Пользователь А передает пользователю Б исходное сообщение вместе с его ЭЦП. После получения этого сообщения пользователь Б при помощи открытого ключа пользователя А расшифровывает данные ЭЦП и сравнивает их с набором символов исходного сообщения. Если значения совпали, то можно сделать утверждение, что данные не были изменены и действительно принадлежат его автору.

СЗИ от вирусов системы.

Вирус системы – это программа небольшого размера, наносящая вред системе. Она способна самостоятельно создавать свои копии и внедрять их в исполняемые файлы программ, в различные документы системы и распространять их по каналам связи.

Компьютерные вирусы в зависимости от областей обитания делятся на:

- программные;
- загрузочные;

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		14

- макровирусы;
- сетевые.

В качестве источников вирусов могут выступать переносные носители информации и системы связи. Наиболее популярными антивирусными программами являются: Антивирус Касперского 7.0, AVAST, Norton AntiVirus и многие другие.

1.3 Криптографическая защита информации

Для сохранения информации в секрете используют методы шифрование или, другими словами, применяют криптографию. Для осуществления шифрования данных применяют алгоритм или используют устройство, которое реализует в свою очередь определенный криптографический алгоритм.

Криптография – это наука о различных способах защиты информации, к которой необходимо ограничить доступ, то есть необходимо сохранить конфиденциальность данной информации.

Криптографические методы защиты информации – это изменение или преобразование данных для их дальнейшей транспортировки доверенному лицу. Принцип этих методов заключается в следующем. Исходное сообщение зашифровывается, принимая форму закрытого текста (графического изображения), которое называется шифротекстом. Затем, шифротекст отправляется пользователю по защищенному или даже допустимо по незащищенному каналу передачи данных. Получив шифротекст, авторизованный пользователь выполняет обратное преобразование или дешифрование принятого сообщения и восстанавливает исходный текст. Таким образом, неавторизованный человек, который может перехватить шифротекст, не сможет его прочитать, не имея ключа и не имея информации о том, какие криптографические методы защиты данных использовались при шифровании.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		15

Криптографию в целом можно разделить на два класса: симметричные и асимметричные криптосистемы.

Криптография с симметричным ключом.

В криптографии с симметричным ключом (классическая криптография) пользователи используют один и тот же (общий) ключ (который еще называют секретным ключом) для шифрования и затем дешифрования данных. В основном такие алгоритмы используются для обеспечения конфиденциальности данных.

На сегодняшний день существует несколько десятков стандартов шифрования как национальных, так и международных, которые уже ни раз были проверены временем. Симметричные шифры – это DES [3] (Data Encryption Standard), в свое время был создан компанией IBM и используется в США в качестве федерального стандарта. IDEA [4] (International Data Encryption Algorithm) еще один алгоритм, позволяющий зашифровывать данные принадлежит к классу международных алгоритмов. Среди шифров с открытым ключом можно также отметить MD 20899, ISO / IEC / DIS 9594-8, X.509.

Криптосистема NTRUEncrypt является криптосистемой с открытым ключом, поэтому стоит рассмотреть подробнее этот раздел криптографии.

1.4 Алгоритмы ассиметричного шифрования

Ассиметричное шифрование – это раздел криптографии, в котором, в отличие от симметричных алгоритмов, используется два различных ключа: ключ шифрования и ключ расшифрования или, другими словами, открытый и закрытый ключи. Открытый ключ находится в свободном доступе и может быть предоставлен любому желающему, в то время как закрытый ключ должен храниться в строжайшем секрете.

Ассиметричное шифрование используется в двух направлениях:

- для создание электронной подписи (ЭП);

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		16

– для обеспечения защищенности и конфиденциальности обмена информацией.

В первом случае, документ шифруется при помощи закрытого ключа владельца подписи и любой, кому известен открытый ключ может проверить, действительно ли подпись является подлинной. Этот способ также может быть использован для обеспечения аутентификации.

Во втором случае, сообщение шифруется открытым ключом получателя, а тот, в свою очередь, имеет возможность его расшифровать своим закрытым ключом.

Определим набор требований, которым должен соответствовать стойкий алгоритм шифрования с открытым ключом (Диффи и Хеллман):

1. Вычислительно просто создать пару открытый и закрытый ключи.
2. Вычислительно просто создать шифротекст, имея соответствующий открытый ключ и сообщение для передачи.
3. Вычислительно просто расшифровать сообщение, используя необходимый закрытый ключ.
4. Вычислительно невозможно определить закрытый ключ по известному открытому.
5. Вычислительно невозможно восстановить исходное сообщение, имея шифротекст и открытый ключ.

На рисунке 1.1 изображены основные шаги шифрования с открытым ключом:

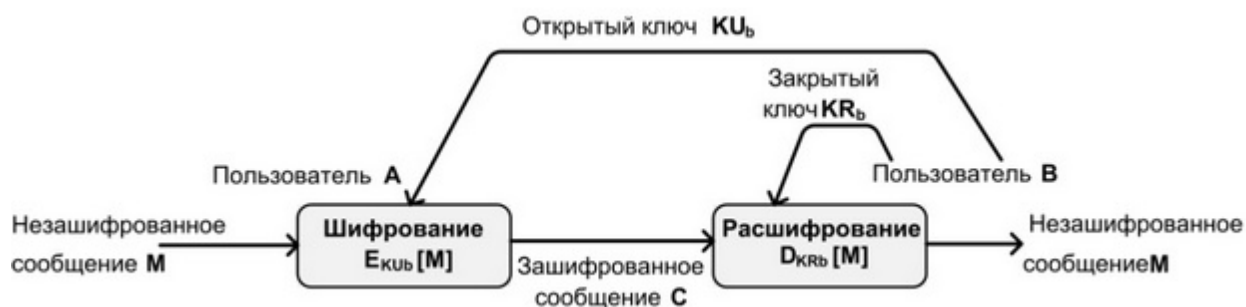


Рисунок 1.1 – Схема асимметричного шифрования

Пользователь А хочет передать сообщение М пользователю В:

1. Пользователь В создает пару ключей: открытый (KU_b) и закрытый (KR_b) соответственно.
2. Пользователь В передает пользователю А свой открытый ключ (KU_b).
3. Пользователь А зашифровывает исходное сообщение М открытым ключом (KU_b) и передает его пользователю В.
4. Когда пользователь В его получает, он расшифровывает его при помощи своего секретного ключа (KR_b). Никто, кроме В, не сможет расшифровать сообщение, так как секретный ключ хранится у пользователя В в строгом секрете.

На рисунке 1.2 схематично изображен алгоритм создания и проверки ЭП:



Рисунок 1.2 – Жизненный цикл ЭП

Пользователь А хочет передать сообщение с подписью пользователю В:

1. Пользователь А создает два ключа: открытый (KU_b) и закрытый (KR_b) соответственно.
2. Пользователь А передает пользователю В свой открытый ключ (KU_b).
3. Пользователь А создает подпись $SignKRa[M]$ исходного текста М, при помощи своего секретного ключа (KR_b).
4. После получения сообщения, пользователь В проверят подпись $VerKUa[M]$ при помощи открытого ключа (KU_b). Никто, кроме А, не сможет подписать сообщение, так как закрытый ключ А хранит в секрете.

5. Подпись пользователя А будет достоверной так долго, как долго она будет надежно храниться в секрете. Не обладая правами доступа к секретному ключу, невозможно будет изменить сообщение, а это значит, что будет обеспечена аутентификация пользователя, который отправил сообщение и целостность передаваемой информации.

Стоит отметить, что подпись не обеспечивает конфиденциальности передачи данных. Да, сообщение изменить не получится, но вполне возможно его «подсмотреть».

1.5 Асимметричная криптосистема RSA

Криптосистема RSA [5] (Rivest, Shamir, Adlema) – криптографический алгоритм с открытым ключом, который основывается на сложности решения задачи факторизации больших целых чисел.

Алгоритм создания открытого и секретного ключей следующий:

1. Произвольным образом пользователь выбирает два больших простых числа p и q .

2. Затем вычисляет число $n = p * q$.

3. Вычисляет функцию Эйлера от числа n

$$\varphi(n) = (p - 1)(q - 1).$$

4. Выбирает число e ($1 < e < \varphi(n)$). На данное число накладывается условие взаимной простоты со значением функции Эйлера. В качестве e рекомендуется выбирать простое число, содержащее сравнительно малое количество единиц в двоичном представлении. Например, числа Ферма, такие как 17, 257, 65537 и т.д.

5. Следующим шагом пользователь вычисляет число d такое, что будет выполняться сравнение

$$d * e \equiv 1 \pmod{\varphi(n)}.$$

6. Пара (e, n) будет являться открытым ключом.

7. Секретный ключ – (d, n) должен храниться в тайне.

Теперь рассмотрим этапы шифрования:

1. Имеется некоторый открытый текст $m \in Z_n$.

2. Шифрование осуществляется по формуле

$$c = m^e \pmod{n}.$$

Расшифрование, то есть получение исходного сообщения m из шифротекста, осуществляется при помощи секретного ключа по формуле

$$m = c^d \pmod{n}.$$

1.6 Введение в теорию квантовых компьютеров

1.6.1 Теория квантовой информации

Классическая теория информации в сочетании с квантовой физикой образуют такое понятие, как теория квантовой информации. Наибольший интерес в условиях растущей популяризации информационных технологий представляют разделы, которые относятся к квантовым вычислениям и соответственно к квантовой криптографии. На сегодняшний день специалисты значительно продвинулись в данной области и получили довольно серьезные теоретические результаты. Полученные знания и решения нашли свое практическое применение в разделах квантовой криптографии. Рынок информационных технологий уже пополнил свои ряды устройствами, которые реализуют протоколы квантовой криптографии, применимые в реальных коммуникационных системах.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		20

Постквантовая криптография объединяет такие криптографические алгоритмы, которые способны сохранить свою эффективность после появления квантовых компьютеров.

Когда речь заходит о квантовом компьютере, то под ним обычно понимают вычислительное устройство, использующее понятие суперпозиции квантовых состояний и квантовой запутанности для передачи и последующей обработки данных. Обычные компьютеры оперируют битами, которые способны принимать одно значение либо нуля, либо единицы, в отличие от квантового компьютера, который использует кубиты способные принимать значение и нуля, и единицы одновременно. В результате достигается огромное превосходство над обычными компьютерами, за счет того, что квантовый компьютер способен обрабатывать все существующие комбинации одновременно.

1.6.2 Алгоритм Питера Шора

Некоторые специалисты начинают историю квантового компьютера вместе с статьей, написанной в 1982 году Ричардом Фейнманом. Далее в середине восьмидесятых годов появились работы А. Дойча. В них были построены формальные модели квантового компьютера – например, квантовая машина Алана Тьюринга. Дальше статья Питера Шора [2], в которой он построил алгоритм, который можно выполнить на квантовом компьютере. Это алгоритм разложения двух чисел на простые множители. Данная статья [2] вызвала огромный резонанс в обществе, так как данный алгоритм позволял с легкостью взламывать любой криптографический шифр. Этот алгоритм был разработан ещё в 1994 году и позволяет раскладывать числа на простые множители. Если вы умеете быстро разлагать большие числа на множители, то вам подвластны любые коды и шифры. Именно поэтому на рубеже 21 века, вокруг этой темы было очень много споров. Фактически Шор создал ярко

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		21

выраженную угрозу всем, в том числе и банковским счетам в интернет банках. Если на обычных компьютерах факторизация 129 – разрядного числа потребует 500 MIPS (Million Instructions Per Second, что в переводе означает «миллион команд в секунду»), или восемь месяцев непрерывной работы системы из 1600 рабочих станций, объединенных через Интернет, а при числе разрядов порядка 300 это время существенно превзойдет возраст Вселенной – даже если работать одновременно на всех существующих в мире вычислительных машинах, то алгоритм Шора ускоряет этот процесс в миллионы раз. В 2001 году алгоритм был успешно проверен на всего 7q – bit компьютере компании IBM, что неопровержимо доказало его возможности. Рассмотрим алгоритм точнее.

Стандартный компьютер занимается операцией подбора. Данная операция достаточно эффективна, если у нас немного вариантов. Так, например, знаменитый компьютер DeepBlue, который в 1997 году обыграл не менее знаменитого игрока в шахматы Гарри Каспарова [6]. Представьте себе шахматную доску, у вас есть возможность совершить определенное количество ходов, с определенными результатами, проще говоря, вы можете перебрать все ходы подбором, но тогда у вас будет огромное количество вычислений. Поэтому ещё не была придумана машина, способная обыграть человека в игру Го, так как вариантов ходов там значительно больше. Алгоритм Шора позволяет и это.

Что касается криптографических систем с открытым ключом, то при помощи алгоритма Шора становится возможным их взлом. Например, RSA [5] основывается на вычислительной сложности задачи факторизации больших чисел, и использует открытый ключ M , который представляет собой произведение двух больших простых чисел. Чтобы осуществить взлом криптосистемы – достаточно найти множители числа M . Наиболее популярными из известных и доказанных алгоритмов являются метод квадратичных форм Шенкса [7] и алгоритм Полларда — Штрассена [8],

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		22

которые работают за время порядка $M^{\frac{1}{4}}$. Лидером среди вероятностных алгоритмов по праву считается специальный метод решета числового поля, который способен с вероятностью 0.5 найти делитель числа за время $\exp\left(\left(\frac{32}{9}\log M\right)^{\frac{1}{3}} * (\log\log M)^{\frac{2}{3}}\right)$. Используя возможности квантового компьютера алгоритм Шора способен решить задачу факторизации не просто за полиномиальное время, а даже за время, которое требуется для проведения операции умножения целых чисел. Таким образом, с изобретением квантового компьютера становятся бесполезными многие криптографические системы, в том числе и RSA. По этой причине была создана система NTRUEncrypt более стойкая к атакам квантовых компьютеров, за счет трудностей поиска кратчайшего вектора решетки. По состоянию на середину 2010-х годов неизвестно ни одного квантового алгоритма, способного справиться лучше обычного компьютера.

1.6.3 Квантовые гейты

Фейнман в 1982 году выдвинул идею моделирования квантового компьютера. Она довольно проста и заключается в построении компьютера на базе квантовых элементарных ячеек вместо классических. Кубиты – квантовые биты, поведение которых описывается законами квантовой механики, обеспечивают преимущество квантового компьютера перед обычным, например в скорости и в возможности параллельного вычисления.

1.6.4 Отличие кубитов от классических битов

Элементарные ячейки в классическом компьютере представляют собой биты, принимающие значения единицы или нуля. В регистр (набор битов) информация подается и обрабатывается в двоичном виде. Один бит может

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		23

находиться в одном из двух возможных (базисных) состояниях: нуль и единица. Путем перебора базисных состояний появляется возможность закодировать произвольное двоичное число. В качестве примера рассмотрим систему, содержащую три бита, в ней можно записать, например, одну из восьми существующих последовательностей.

Будем использовать специальные обозначения бра/кет для более компактного описания состояний и преобразований квантовой системы. Эти обозначения были введены Дираком [9] еще на самой заре появления на свет квантовой механики в качестве более удобного средства оперирования векторами. Квантовые состояния зачастую описываются кет-векторами $|x\rangle$ и обозначают вектор-столбцы. Внутри скобок, как описывает Дирак [9], помещается индекс состояния системы, или, другими словами, набор величин, определяющих эту систему. Вектор-строку обозначает бра-вектор $\langle y|$.

Как уже упоминалось выше, квантовый компьютер использует кубиты, образующих квантовую систему, которая по аналогии с битом содержит два базисных состояния $|0\rangle$ и $|1\rangle$, отличие заключается в том, что кубит способен принимать любое из суперпозиционных состояний $|q\rangle = a|0\rangle + b|1\rangle$, где a, b – комплексные числа, $|a|^2 + |b|^2 = 1$. Получается, что кубит может принимать невообразимо огромное количество значений, но на выходе в итоге получим либо состояние $|0\rangle$ с вероятностью $|a|^2$, либо $|1\rangle$ с вероятностью $|b|^2$.

Именно квантовое перепутывание, а не непрерывная природа суперпозиционных состояний в системе кубитов является самым важным отличием от обычных классических битов. Квантовая механика определяет размерность пространства состояний системы как произведение размерностей отдельных подсистем пространств состояний. Система из N кубитов имеет 2^N базисных состояний. Если рассмотреть любое состояние из N кубитов $(a_1|0\rangle + b_1|1\rangle)(a_2|0\rangle + b_2|1\rangle) \dots (a_n|0\rangle + b_n|1\rangle)$, то оно будет содержать абсолютно все комбинации длины N , состоящие из нулей и единиц. Например, для $N=3$, как и в примере, рассмотренном выше, все восемь двоичных чисел одновременно

смогут быть закодированы всего лишь в трех кубитах. Достигнуть этого становится возможным при помощи квантового перепутывания. Нелокальные корреляции в системе кубитов и обеспечивают экспоненциально большое вычислительное пространство и параллелизм квантовых вычислений.

Также гейты можно разделить на одно- и многокубитные в зависимости от числа кубитов, которые используются. По аналогии с обычным регистром существует квантовый, состоящий из набора кубитов. Гейт переводит регистр из одного состояния в другое. Описывается это действие гейта на регистр таким образом: $G|R\rangle=|R'\rangle$.

Гейты – линейные операции: $G(|p\rangle+|g\rangle)=G|p\rangle+G|g\rangle$.

1.6.5 Квантовые вычисления

Для обеспечения большого числа параллельно выполняющихся процессов модель квантовых вычислений использует несколько относительно простых правил преобразования входной информации. Иными словами, становится возможным за один проход функции вычислить ее значения для всех аргументов.

Данная функция применяется к входным данным, рассмотрим ее подробнее. Как уже было сказано на вход подаются данные, которые представляют собой суперпозицию всех возможных значений аргумента, следовательно, функция должна каким-то образом принять и суметь обработать такую последовательность. В квантовых моделях функция представляет из себя эрмитовую матрицу, то есть такую, которая имеет свою эрмитово-сопряженную, при этом произведение этих матриц даст в итоге единичную. Матрица должна быть именно эрмитовой – это и есть одно единственное ограничение, накладываемое на матрицу. Сопряженную матрицу можно получить в два шага, первый шаг – транспонирование исходной матрицы и второй шаг – вместо исходных элементов подставляем их

комплексно-сопряженные. Тогда если умножить получившуюся матрицу на вектор квантового регистра, получим в итоге еще один квантовый регистр, такой что сумма квадратов коэффициентов при квантовых состояниях будет равна единице.

Рассмотрим еще одну операцию, которая имеет место быть в модели квантовых вычислений – измерение. Эта операция дает возможность получить конкретное значение кубита из суперпозиции путем измерения вектора. Вероятность получения какого-либо значения будет равна квадрату комплексного коэффициента. Так как сумма вероятностей получения всех возможных значений случайной величины равняется единице и так как при выполнении операции измерения получено конкретное значение, то сумма квадратов комплексных коэффициентов будет также равняться единице.

Таким образом, если имеется N кубитов, то число 2^N – это количество комплексных значений, которые можно будет одновременно вычислить. На выходе получится вектор, который представляет собой результаты всех вычислений. Мощь квантовых вычислений заключается именно в этом. Существует и ограничение этой модели, из-за того факта, что получить можно только одно значение, причем каждый раз разное, зависит это от распределения вероятностей.

Сам квантовый алгоритм можно кратко описать следующим образом. Первым делом вычисляется суперпозиция всех возможных значений входного параметра и передается функции для обработки. Затем анализируя результаты её работы делается вывод о свойствах этой функции.

1.6.6 Квантовые алгоритмы

Квантовые вычисления описаны во многих источниках и практически в каждом из них найдется упоминание про несколько алгоритмов, позволяющие наглядно продемонстрировать всю силу вычислительной модели. Данный

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		26

раздел посвящен краткому и поверхностному описанию двух таких алгоритмов, которые показывают основы квантовых вычислений.

Алгоритм Дойча.

Одним из самых первых алгоритмов квантовых вычислений считается алгоритм Дойча, который преследует цель демонстрации самой сути и эффективности таких вычислений. Алгоритм реализует решение задачи, которая абстрагирована от реальности, но именно на примере этой задачи можно показать главный принцип, составляющий базу модели.

Рассмотрим функцию, на вход которой поступает информация в один бит и в результате обработки этого бита получаем другой бит. Вариантов подобных функций всего может быть четыре штуки. Первые две представляют собой постоянные функции (константные), то есть такие, которые возвращают всегда только одно значение, первая возвращает ноль, вторая – единицу. Две оставшиеся функции являются сбалансированными, а это означает, что в одинаковом количестве сценариев будет возвращен ноль и единица. Остается открытым вопрос о том, как определить за один проход функции к какому классу она принадлежит.

Этот вопрос будет неразрешим в системах классического вычисления в силу того, что функцию необходимо будет вызвать минимум два раза и затем провести сравнение полученных результатов. Решение проблемы предоставляет модель квантовых вычислений, в которой эта задача решается одним вызовом функции.

Как уже было сказано в разделе 1.6.5, на начальном этапе для всех возможных входных значений подготавливается равновероятностная суперпозиция. Существует специальная функция под названием гейт Адамара и так как на вход подается один кубит, то с помощью этого гейта подготавливается равновероятностная суперпозиция входного кубита. На следующем шаге опять используется гейт Адамара, который выполняет преобразование по такому принципу: если на вход была принята

равновероятностная суперпозиция, то в зависимости от фазы в которой она находится принимается решение о переводе ее назад в состояние $|0\rangle$ или $|1\rangle$. Затем в ход вступает функция измерения кубита, если в результате ее работы получаем $|0\rangle$, то исходная функция относится к постоянным (константным), если же результат равен $|1\rangle$, то очевидно относим её к сбалансированным.

Как уже было сказано в разделе 1.6.5, получить все значения функции, к сожалению, нет возможности, но вот сделать выводы о каких-то её свойствах можно вполне. Инструментом в поиске таких свойств функции как раз и является задача Дойча. Само свойство довольно простое и заключается в том, что если функция константная, то результатом сложения всех возможных её входных значений по модулю двух всегда будет нуль. В случае, если функция является сбалансированной, результатом сложения, наоборот, будет всегда единица. В результате работы алгоритма Дойча был получен именно этот результат. Нет информации о том какое конкретно значение возвратила функция на равновероятностной суперпозиции входного кубита, однако есть информация о том, что это значение представляет из себя суперпозицию результатов, и теперь если её модифицировать определенным образом, то можно будет сделать вывод о свойстве этой функции.

Алгоритм Гровера.

Вторым важным алгоритмом в модели квантовых вычислений является алгоритм Гровера, которая демонстрирует выигрыш таких вычислений в сравнении с классической моделью и предлагает решение более близкой к реальности задачи. Сам Лов Гровер называет свой алгоритм поиском иглы в копне сена. Основан этот алгоритм на принципе амплификации, который составляет базу всех квантовых вычислений. Рассмотрим алгоритм Гровера подробнее.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		28

Создается специальная функция, задача которой состоит в изменении фазы у состояния $|1\rangle$. Под фазой квантового состояния в суперпозиции понимают знак у коэффициента перед этим состоянием.

Алгоритм Гровера дает решение задачи нахождения определенного элемента, который удовлетворяет каким-то требованиям, во множестве неупорядоченных данных. И справиться с решением этой задачи алгоритм может эффективнее, чем простым перебором. Для наглядности, простым перебором данная задача решается за $O(N)$ вызовов функции, в то время как алгоритм Гровера вычисляет нужный элемент за $O(\sqrt{N})$ вызовов функции.

Рассмотри шаги алгоритма Гровера:

1. Как и в алгоритме Дойча, на начальном шаге подготавливается равновероятностная суперпозиция входных кубитов.

2. Выполнение итерации. Она состоит в применении шаг за шагом функции поиска, которая осуществляет поиск согласно требованиям, и затем в применении так называемого гейта диффузии, который выполняет преобразование коэффициентов перед квантовыми состояниями, путем их вращения вокруг среднего. За счет этого происходит упомянутая выше амплификация, то есть увеличение амплитуды искомого значения. Нюанс заключается в том, что применить такую итерацию следует несколько раз, а если точнее, то фиксированное количество раз – $(\sqrt{2n})$, в противном случае алгоритм возвратит неверный результат.

3. Выполнение измерения. Измеряется входной квантовый регистр (набор кубитов) и после этого шага с высокой вероятностью будет получен конечный результат работы алгоритма. Если есть необходимость в увеличении достоверности результата, то этот алгоритм прогоняется еще несколько раз и определяется в совокупности вероятность верного результата.

1.6.7 Применение квантовых гейтов

Квантовая модель вычислений позволяет:

- осуществить разложение некоторого числа M на множители [1] за время равное $O(\lg^3 M)$;
- осуществить полный перебор за время $\sqrt{n}$ (алгоритм Гровера);
- осуществить дискретный алгоритм нахождения логарифма за полиномиальное время.

В XXI веке одной из самых глобальных задач современной науки является построение реального квантового компьютера в виде некоторого физического устройства.

В 2005 году коллективом под руководством Ю. Пашкина совместно с японскими специалистами был спроектирован и собран двухкубитный квантовый процессор на основе сверхпроводящих элементах.

В ноябре 2009 года физикам из США впервые удалось собрать программируемый квантовый компьютер, состоящий из двух кубит.

11 мая 2011 года был создан компьютер D-Wave One, работающий на базе 128-кубитного процессора.

В декабре 2012 года представлен новый процессор Vesuvius, который объединяет 512 кубит.

1.7 Кратчайший вектор решетки

«Постквантовая криптография» содержит огромный ряд криптографических примитивов и протоколов, основанных на сложности решения задач теории решеток:

- ассиметричные криптосистемы: Айтая-Дворка, Реджева, Джентри, NTRU;

- криптографических хеш-функциях: Айтая [10][11], LASH [12], SWIFFT [13], SWIFFTX [14];
- протоколах цифровой подписи: Гольдвассера-Гольдштейна-Халеви (GGH) [15], Джентри-Пейкерт-Вейконтанатана [16], Миссиансио-Вадхена [17], Миссиансио-Любашевского [18];
- протоколах защищенного обмена данными и идентификационных схемах: Пейкерт-Вейконтанатана [19], Любашевского [20].

В связи с этим, стоит отметить значительную необходимость научного развития в направлении изучения задач и различных их свойств в теории решеток, так как именно они представляют собой один из самых основных этапов построения протоколов и примитивов «постквантовой криптографии». Исследование в этой области представляет интерес в связи с тем, что при определенных параметрах задачи теории решеток становятся постквантовыми, то есть обладают криптостойкостью к различным атакам квантовых алгоритмов.

После открытия алгоритма создания фотонного квантового компьютера рядом ученых во главе с Джереми О'Брайеном, необходимость в изучении постквантовых задач в разы возрастает [21][22].

Решетка представляет собой дискретную аддитивную подгруппу, которая задается на множестве $\mathbb{R}^n$, или, другими словами, решетка L может быть представлена в виде:

$$L(b_1, \dots, b_n) = \left\{ \sum_{i=1}^n x_i b_i : x_1, \dots, x_n \in \mathbb{Z} \right\}.$$

где число n обозначает количество линейно независимых векторов, которые называются базисными векторами $\{b_1, \dots, b_n\} \subset \mathbb{R}^m$, в Евклидовом пространстве размерностью m и рангом равным n (рис. 1.3).

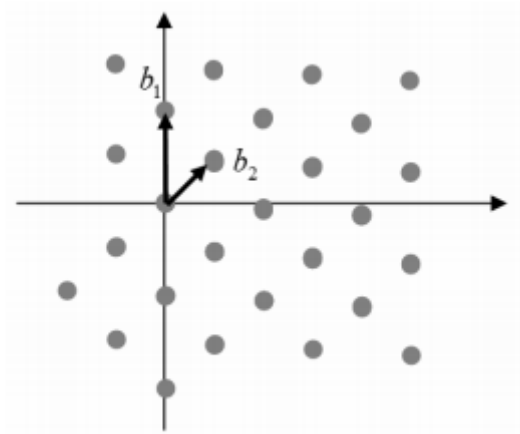


Рисунок 1.3 - Решетка L образована базисными векторами b_1 и b_2

Решетка может иметь большое число различных базисов

$$L = \sum_{i=1}^n a_i * Z.$$

На рисунке 1.4 изображена та же решетка, что и на рисунке 1.3, только с другим базисом – a_1 и a_2 .

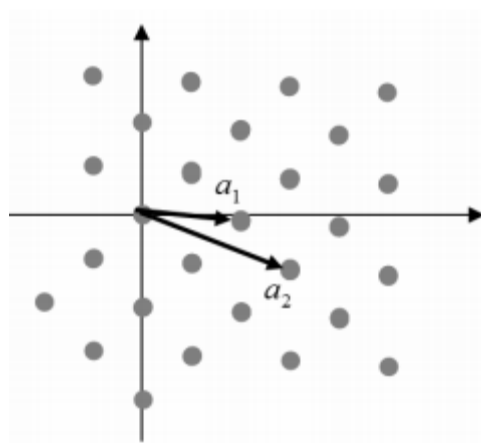


Рисунок 1.4 - Решетка L образована базисными векторами a_1 и a_2

Базисы решетки образуют параллелепипеды, которые еще называют фундаментальными, на рисунках 1.5 и 1.6 показаны примеры таких параллелепипедов.

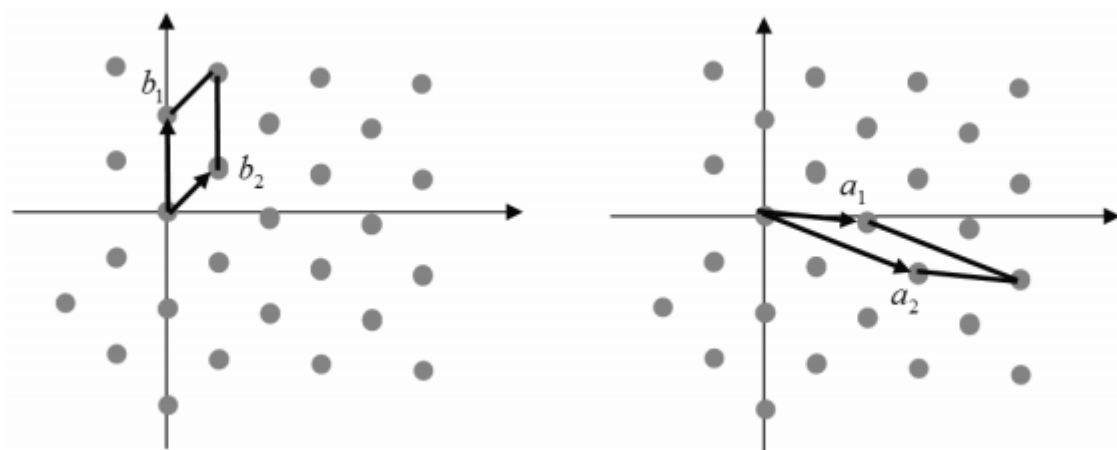


Рисунок 1.5,.1.6 – Образованные базисами решетки параллелепипеды

В двумерном пространстве площади, а в многомерном – объемы параллелепипедов, образованных всеми возможными базисами одной решетки L , будут равны между собой. Кратчайший вектор решетки – это вектор, длина которого удовлетворяет формуле

$$\lambda(L) = \min ||x - y|| = \min ||x||.$$

На рисунке 1.7 изображен кратчайший вектор решетки L .

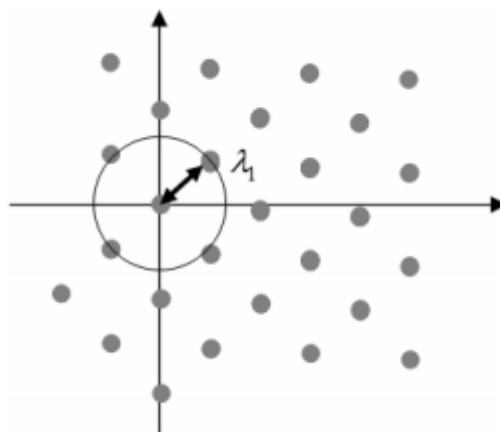


Рисунок 1.7 – Кратчайший вектор решетки L

В случае, если решетка представлена, как L^n , то есть имеет размерность n , то данное понятие можно расширить и тогда в многомерном случае это будет i -й последовательный минимум решетки $\lambda_i(L)$, представляющий собой

наименьший радиус окружности в двумерном пространстве и шара в многомерном, внутри которого находятся i -линейно независимых векторов

$$\lambda_i^p(L) = r,$$

$$r \in \mathbb{R} : \exists v_i \in L, \max \|v_i\| \leq r,$$

где v_i – это один из i -линейно независимых векторов, рисунок 1.8.

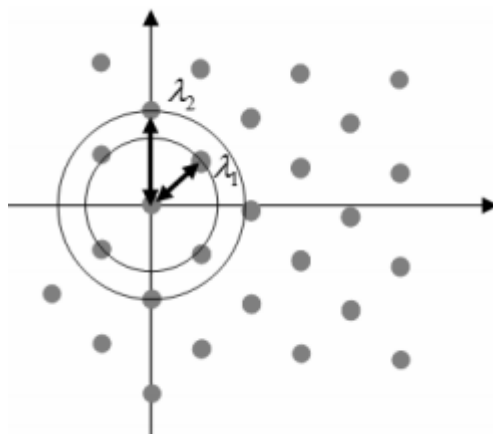


Рисунок 1.8 - Кратчайший базис решетки L в двумерном пространстве

Кратчайшему вектору будет соответствовать первый последовательный минимум в решетке.

Существует такое понятие, как идеальная решетка. Решетка называется идеальной, если в результате операций сложения и умножения векторов в решетке получается вектор, принадлежащий так же этой решетке. Рассмотрим кольцо многочленов $R = \mathbb{Z}[x] / f(x)$, коэффициенты которых являются целыми числами, а сами многочлены взяты по модулю $f(x)$ – многочлен степени n . Такое отношение является решеткой, так как оно изоморфно над $\mathbb{Z}^n$ и аддитивные группы являются подгруппами в R . Этот класс решеток так же называют идеальными решетками.

Ниже будут перечислены задачи теории решеток, которые лежат в основе построения рассмотренных выше криптографических системах [17]:

1. Задача SVP. Нахождение кратчайшего вектора решетки, по известному базису решетки, рисунок 1.9.

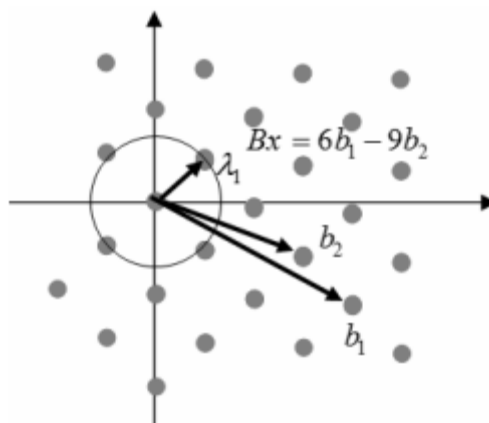


Рисунок 1.9 –SVP-задача

2. Задача SVP_γ . Приближенный поиск кратчайшего вектора. Нахождение вектора $\bar{b}$ не равного нулю, по известному базису и числу $\gamma > 0$, такого, что $\bar{b} \in BZ^n \setminus \{0\} : \|\bar{b}\|_p \leq \gamma * \lambda_1^p(L)$, рисунок 1.10.

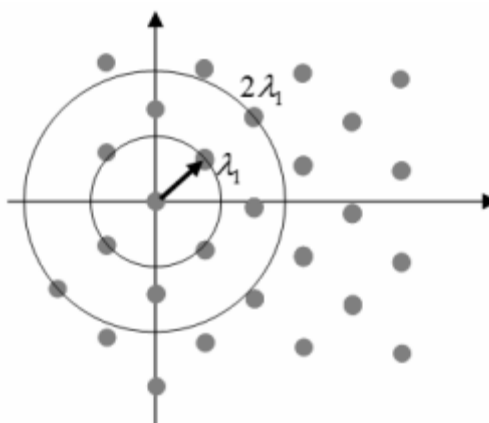


Рисунок 1.10 - SVP_γ - задача

3. Задача CVP. Нахождение ближайшего вектора, по известному базису и некоторому вектору j не принадлежавшему, рисунок 1.11.

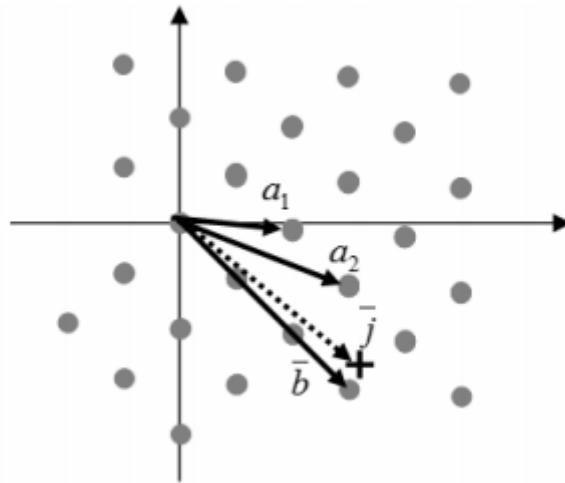


Рисунок 1.11 – CVP - задача

4. Задача CVP_1^p . Нахождение по известному базису, числу $\gamma > 0$ и вектору $\bar{j}$ не принадлежавшему решетке, вектора не равного нулю, такого что $\bar{b} \in BZ^n: \|\bar{j} - \bar{b}\|_p \leq \gamma \lambda_1^p(L)$.

5. Задача $SIVP_1^p(n, \gamma)$. Приближенное нахождение кратчайших линейно независимых векторов, рисунок 1.12.

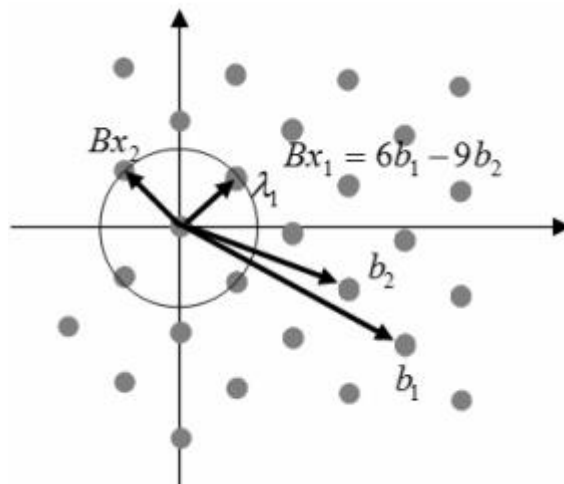


Рисунок 1.12 – SIVP - задача

1.8 Выводы по первой главе

1. Рассмотрены и кратко описаны основные положения безопасности информационных систем, источники угроз ИБ и средства защиты информации.

2. Рассмотрен один из классов криптографических систем – асимметричные криптосистемы, приведен пример из этого класса в виде криптосистемы RSA.

3. Дано описание задачи теории решеток, на основе которой разработан алгоритм NTRUEncrypt и за счет которой обеспечивается ее стойкость к возможным квантовым атакам, задача о нахождении кратчайшего вектора решетки.

4. Представлена идея квантового компьютера, приведены понятия теории квантовых вычислений, а также кратко дано описание основной идеи двух алгоритмов, которые демонстрируют различные базовые принципы квантовых вычислений – алгоритм Дойча и алгоритм Гровера.

2 Алгоритмизация программных средств

2.1 Система шифрования NTRUEncrypt

Криптографическая система с открытым ключом NTRUEncrypt использует операции над кольцом $Z[X] / (X^N - 1)$ многочленов степени, не превосходящей $N - 1$

$$a = a_0 + a_1 * X^1 + a_2 * X^2 + \dots + a_{N-1} * X^{N-1},$$

где $a_0, a_1, a_2 \dots a_{N-1}$ – целые числа.

Операции сложения и умножения производятся как обычно, за исключением того, что X^N заменяется на 1, X^{N+1} заменяется на X^1 , X^{N+2} заменяется на X^2 и т.д.

Криптосистема определяется рядом параметров, основные из которых: N , p и q . Для сохранения стойкости алгоритма необходимо, чтобы параметры p и q были взаимнопростыми.

Чтобы обеспечить высокую стойкость алгоритма к различным атакам, рекомендуется использование следующих параметров, рисунок 2.1.

Обозначение	N	q	p	df	dg	dr	Гарантированная стойкость
NTRU167:3	167	128	3	61	20	18	Умеренный уровень стойкости
NTRU251:3	251	128	3	50	24	16	Стандартный уровень стойкости
NTRU503:3	503	256	3	216	72	55	Высочайший уровень стойкости
NTRU167:2	167	127	2	45	35	18	Умеренный уровень стойкости
NTRU251:2	251	127	2	35	35	22	Стандартный уровень стойкости
NTRU503:2	503	253	2	155	100	65	Высочайший уровень стойкости

Рисунок 2.1 – Рекомендованные параметры

2.1.1 Алгоритм генерации ключей системы

Боб хочет передать сообщение Алисе, для этого ему необходимы открытый и закрытый ключи. Поэтому он случайным образом выбирает два «маленьких» полинома f и g из кольца усеченных многочленов R . «Малость» полиномов означает, что относительно произвольного полинома по модулю q , в котором коэффициенты равномерно распределены, у малого полинома они будут много меньше q . Для определения малости полиномов используются числа df и dg , которые Боб выбирает самостоятельно:

- полином f будет иметь df коэффициентов равных единице, $(df-1)$ коэффициентов равных минус единице и остальные равные нулю;
- полином g будет иметь dg коэффициентов равных единице, столько же коэффициентов равных минус единице и остальные равные нулю.

Боб должен хранить выбранные полиномы в тайне, так как любой, кому они станут известны, сможет расшифровать сообщение.

Следующим шагом Боб вычисляет обратные полиномы f_p и f_q по модулю p и q соответственно, такие, что

$$f * f_p = 1(\text{mod } p) \text{ и } f * f_q = 1(\text{mod } q).$$

В случае, если обратных полиномов не будет существовать, то Бобу нужно вернуться на шаг назад и еще раз выбирать полином f .

Пара (f, f_p) является секретным ключом, а открытый ключ h Боб рассчитывает согласно формуле

$$h = p * f_q * g (\text{mod } q).$$

2.1.2 Алгоритм шифрования сообщения

Алиса теперь знает открытый ключ h и может отправить любую зашифрованную информацию Бобу. Чтобы это осуществить Алисе необходимо представить текст сообщения в виде полинома m , коэффициенты которого приведены по модулю p и которые принадлежат диапазону $(-p/2, p/2]$. Затем Алиса случайным образом выбирает некоторый «маленький» полином r , известный как «ослепляющий» полином и вычисляет шифротекст согласно выражению

$$e = (r * h + m)(\text{mod } q).$$

2.1.3 Алгоритм расшифрование шифротекста

Пусть Боб получил от Алисы зашифрованные данные и его нужно получить их содержание, то есть расшифровать. Сперва при помощи своего секретного ключа Боб вычисляет

$$a = f * e (\text{mod } q).$$

Так как Боб вычисляет значение a по модулю числа q , он должен выбрать его коэффициенты из диапазона $(-q/2, q/2]$ и затем вычислить

$$b = a (\text{mod } p).$$

Наконец, Боб используя вторую часть секретного ключа получает исходное сообщение от Алисы

$$c = f_p * b (\text{mod } p).$$

2.1.4 Доказательство работы алгоритма

Для доказательства работы алгоритма достаточно подробнее рассмотреть процесс расшифрования сообщения.

Зашифрованное сообщение от Алисы имеет вид

$$e = (r * h + m)(\text{mod } q).$$

Бобу изначально неизвестны значения r и m , но у него есть закрытый ключ (f, f_p) , которым он может воспользоваться следующим образом

$$\begin{aligned} a &= f * e (\text{mod } q) = (f * (r * h + m))(\text{mod } q) = \\ &= (f * (r * pf_q * g + m))(\text{mod } q). \end{aligned}$$

В результате получаем выражение

$$a = (pr * g + m * f)(\text{mod } q).$$

Уменьшая коэффициенты полинома a по модулю p Боб рассчитывает полином b по формуле

$$b = a (\text{mod } p) = m * f (\text{mod } p).$$

Это справедливо, так как равняется нулю произведение

$$(pr * g)(\text{mod } p) = 0.$$

Открытый текст Бобу все еще неизвестен, и чтобы его получить, нужно умножить полученное на предыдущем шаге значение полинома b на вторую часть секретного ключа и доказать, что полином c в самом деле является исходным сообщением m

$$c = f_p * b (\text{mod } p) = f_p * f * m (\text{mod } p) = m (\text{mod } p).$$

2.2 Модификация алгоритма NTRUEncrypt

Как видно из описания алгоритма, на полином f накладываются следующие требования

- полином f является обратимым по модулю p ;
- полином f является обратимым по модулю q ;
- полином f является «малым» полиномом.

В самом алгоритме гарантировалась обратимость по модулю p и q тем, что если вдруг генерировался полином f такой, что он не обратим по одному из модулей, то он отбрасывался и генерировался следующий, до тех пор, пока таковой не будет найден.

Предлагаемая модификация заключается в том, чтобы заменить полином f на полином вида

$$f = 1 + pF,$$

где F – «малый» полином.

Данный подход имеет ряд преимуществ:

- из выражения выше видно, что полином f всегда обратим по модулю p . Этот факт ускоряет генерацию ключей, так как не требуется дополнительного вычисления f_p ;
- поскольку $f^{-1} = 1 \bmod p$, то при расшифровании не будет требоваться дополнительного умножения на f^{-1} , что ускорит сам процесс расшифровки. Закрытый ключ, в этом случае, будет представлять собой не пару (f, f_p) , а (f) .

Описание алгоритма генерации ключей.

Как и в оригинальном алгоритме, с самого начала Боб выбирает параметры шифрования N , p , q и числа df , dg . Затем он случайным образом выбирает два «маленьких» полинома F и g из кольца усеченных многочленов R .

Вычисляет модифицированный полином f по формуле выше.

Следующим шагом Боб вычисляет обратный полином f_q по модулю q

$$f * f_q = 1(\text{mod } q).$$

В случае, если обратных полиномов не будет существовать, то Бобу нужно вернуться на шаг назад и еще раз выбирать полином f .

Пара (f, f_p) является секретным ключом, а открытый ключ h Боб рассчитывает согласно формуле

$$h = p * f_q * g (\text{mod } q).$$

Описание алгоритма шифрования.

Шифрование остается без изменений, все в точности, как в оригинальном алгоритме NTRUEncrypt.

Алиса хочет отправить сообщение Бобу с помощью открытого ключа h . Чтобы это осуществить Алисе необходимо представить текст сообщения в виде полинома m , коэффициенты которого приведены по модулю p и которые принадлежат диапазону $(-p/2, p/2]$. Затем Алиса случайным образом выбирает некоторый «маленький» полином r , известный как «ослепляющий» полином и вычисляет шифротекст согласно выражению

$$e = (r * h + m)(\text{mod } q).$$

Описание алгоритма расшифрования.

Пусть Боб получил от Алисы зашифрованные данные и его нужно получить их содержание, то есть расшифровать. Сперва при помощи своего секретного ключа Боб вычисляет

$$a = f * e (\text{mod } q).$$

Так как Боб вычисляет значение a по модулю числа q , он должен выбрать его коэффициенты из диапазона $(-q/2, q/2]$ и затем вычислить

$$b = a \pmod{p}.$$

На данном этапе вычисления заканчиваются, получено исходное сообщение от Алисы: $b = m$.

2.2.1 Доказательство работы модифицированного алгоритма

Для доказательства работы алгоритма будем использовать тот же метод, что и в разделе 2.1.4, а именно – рассмотрим сам процесс расшифрования.

Зашифрованное сообщение от Алисы имеет вид

$$e = (r * h + m) \pmod{q}.$$

Боб использует свой закрытый ключ – полином f

$$\begin{aligned} a &= f * e \pmod{q} = (f * (r * h + m)) \pmod{q} = \\ &= (f * (r * p f_q * g + m)) \pmod{q}. \end{aligned}$$

В результате получаем выражение

$$a = (p r * g + m * f) \pmod{q}.$$

Следующим шагом Боб получает полином b , путем уменьшения коэффициентов полинома a по модулю p

$$b = a \pmod{p} = m * f \pmod{p} = (m + m * p * F) \pmod{p} = m \pmod{p}.$$

Таким образом, мы проверили и доказали, что полином b в самом деле является исходным сообщением m .

2.3 Преимущества системы NTRUEncrypt

Комитет США Accredited Standards Committee X9 [23] семь лет назад утвердил использование алгоритма асимметричного шифрования NTRU (NTRUEncrypt). Сам алгоритм был разработан ещё в середине 90-х годов. В сравнении с криптосистемой RSA, он не получил широкого распространения, так как сначала необходимо было повысить криптостойкость и производительность этого шифроалгоритма. На текущий момент все выявленные недостатки уже исправлены, и на практике считается, что NTRUEncrypt работает намного быстрее, чем алгоритм RSA. Этот факт подтверждают сами специалисты RSA Labs, а также независимые исследователи.

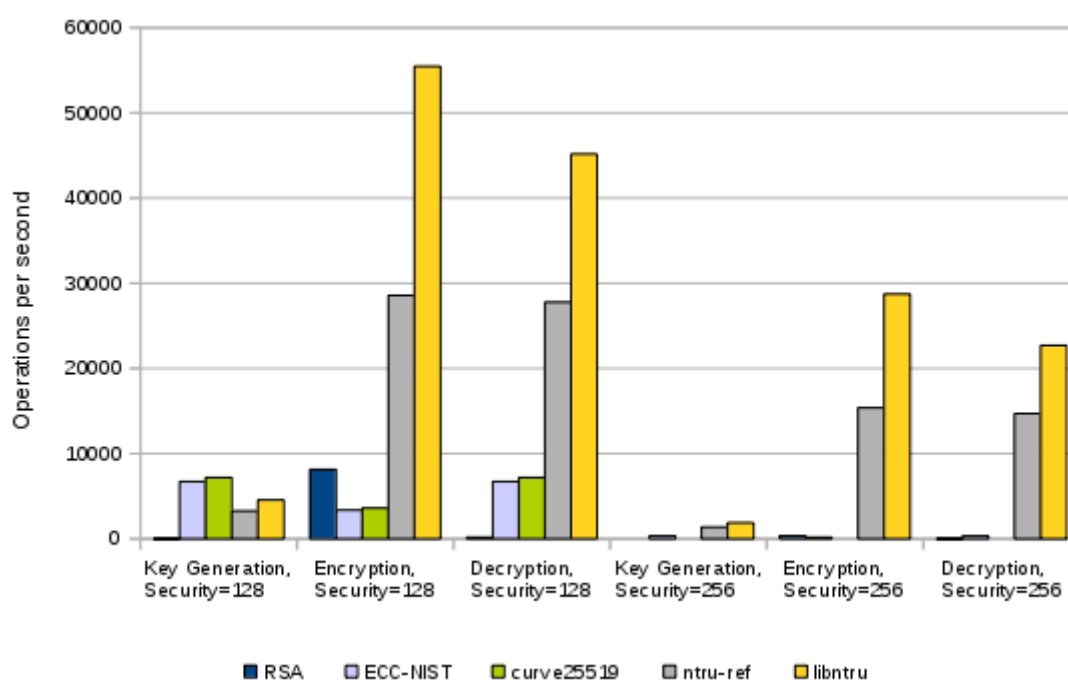


Рисунок 2.2 – Сравнение быстродействия NTRU с другими алгоритмами

Ученые Лёвенского католического университета (Бельгия) провели сравнительный анализ - исследование и обнаружили, что при максимальных настройках безопасности тестирование алгоритма асимметричного шифрования NTRUEncrypt на четыре порядка быстрее алгоритма RSA с

открытым ключом и на три порядка быстрее ECC. На графике (рисунок 2.2) наглядно показано, насколько криптосистема NTRUEncrypt превосходит, по количеству операций в секунду, большинство существующих доквантовых алгоритмов.

К тому же, криптосистема NTRUEncrypt является постквантовой, и имеет выше в разы криптостойкость к различным атакам. И, как уже было сказано выше, за счет отсутствия алгоритма поиска кратчайшего вектора решётки обеспечивается криптостойкость алгоритма.

2.4 Выводы по второй главе

1. Приведены подробные алгоритмы как самой программы NTRUEncrypt, так и ее модификации.

2. Описаны доказательства работы этих двух алгоритмов, а также перечислен ряд преимуществ модификации по сравнению с исходной программой и если обобщить, то вывод получается следующим:

- модификация оптимизирована по параметрам скорости работы алгоритма, так как ускоряется генерация ключей и ускоряется процесс дешифрации;

- код содержит меньше строк, а значит меньше будет весить исходный файл программы, то есть наблюдается экономия внутренней памяти. В дополнение к этому становится проще сам алгоритм, его легче воспринимать;

- уменьшается закрытый ключ, который пользователь хранит в секрете, а следовательно, он меньше весит, то есть опять получаем экономию внутренней памяти.

3. Приведены графические данные, полученные криптологами из Лёвенского католического университета (Бельгия), наглядно демонстрирующие преимущества криптосистемы NTRUEncrypt.

3 Программное конструирование системы NTRUEncrypt

3.1 Выбор языка программирования

В качестве языка программирования был выбран объектно-ориентированный язык программирования C# (C Sharp), относящийся к семье языков с C-подобным синтаксисом. В качестве среды разработки была выбрана Microsoft Visual Studio 2013. Главное преимущество данного программного продукта – разработка приложения с графическим интерфейсом, который позволяет пользователю быстро и легко разобраться в устройстве и схеме работы данного программного продукта.

3.2 Структура разработанных программ

Реализованные программные средства являются однотипными, т.е. они имеют одинаковую структуру, различие между ними заключается только лишь в обработке данных. Их структура представляет собой набор как зависимых, так и независимых обработчиков событий, в нашем случае, событие – нажатие пользователем на кнопку. Так как в основе программ лежит блочный алгоритм, то есть исходное сообщение разбивается на блоки, в зависимости от введенного параметра N, и затем программы работают с блоками данных, то автором было реализовано два режима работы программ. Первый режим является демонстрационным и наглядно показывает работу одного раунда программных средств (работа с одним блоком данных), второй режим является итоговым и работает уже с полноценным файлом входных данных.

Ниже, на рисунке 3.1 приведен схематичный алгоритм работы программы NTRUEncrypt. Почти все модули, кроме первого, являются зависимыми, то есть используют результаты работы какого-то из предыдущих модулей, записанные в массив данных.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		47

На первом подготовительном этапе пользователь должен задать параметры работы системы, значения N , p , q . Параметры df и dg , задающие количество единиц в генерируемых программой полиномах f и g , принимают константные значения равные четырем и трем соответственно.

Второй подготовительный этап заключается в генерации самих полиномов f и g . Затем следует вычисление обратных полиномов k f по модулю p и q , если обратного полинома не существует, то необходимо вернуться назад и сгенерировать новый полином f .

Основными модулями являются:

- модуль генерации ключей. Использует параметры, введенные пользователем на первом шаге;
- модуль шифрования. Использует открытый ключ, полученный программным средством на втором шаге и открытый текст, введенный пользователем побитово или загруженный из файла;
- модуль расшифрования. Использует данные о закрытом ключе так же полученные на втором шаге работы алгоритма и сам шифротекст.

Стоит так же отметить не менее значительный модуль под названием Class1 он был реализован с целью удобства программной реализации, обращение к классу осуществляется через точку Class1., в этот класс вынесены все необходимые операции над полиномами, а именно:

- деление полиномов Class1.Deconv;
- произведение полиномов Class1.Proiz;
- разность полиномов Class1.Raznost;
- сумма полиномов Class1.Sum.

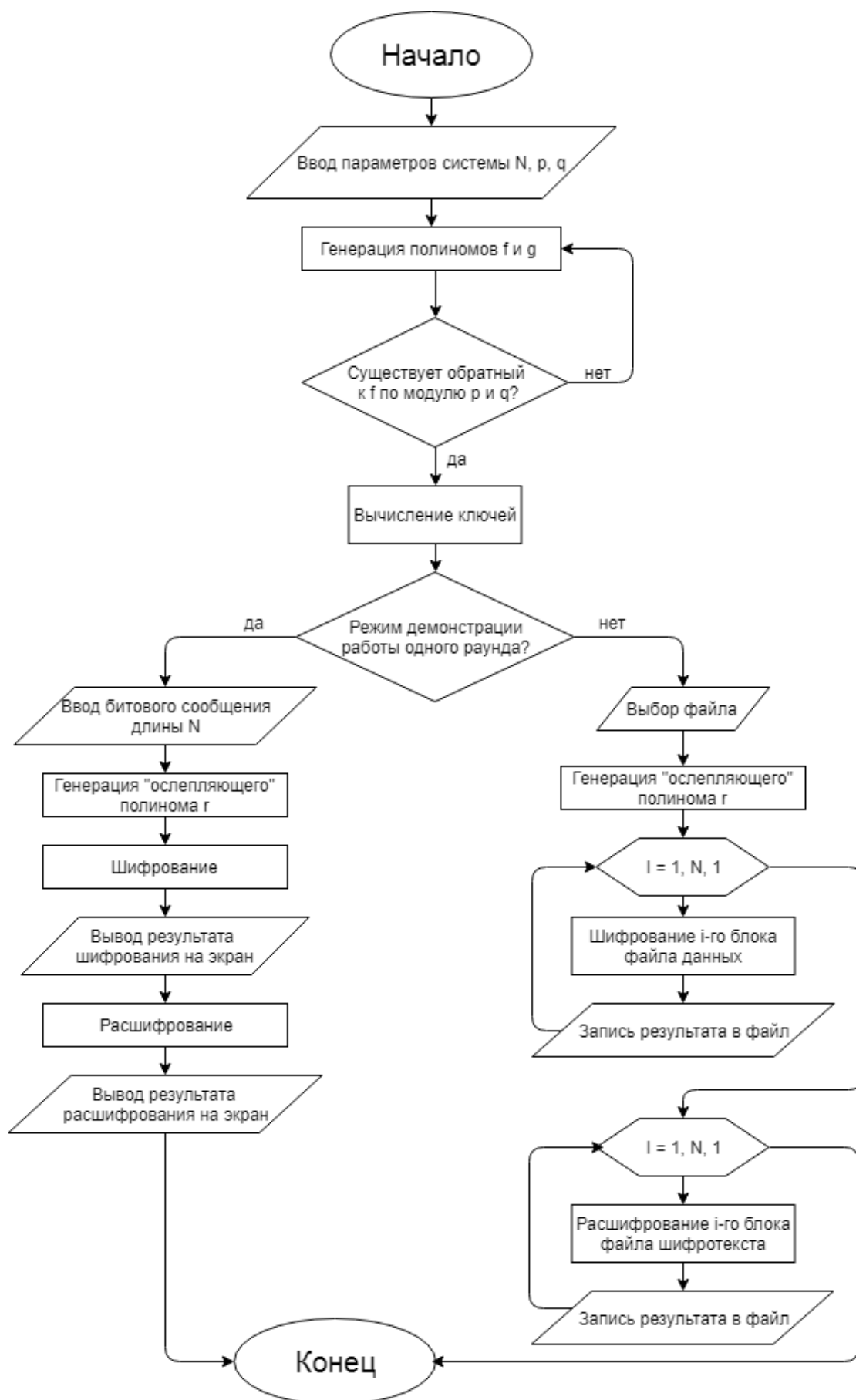


Рисунок 3.1 – Схематичный алгоритм работы программы NTRUEncrypt

3.3 Программная реализация алгоритма NTRUEncrypt

Общий вид диалогового окна программы показан на рисунке 3.2.

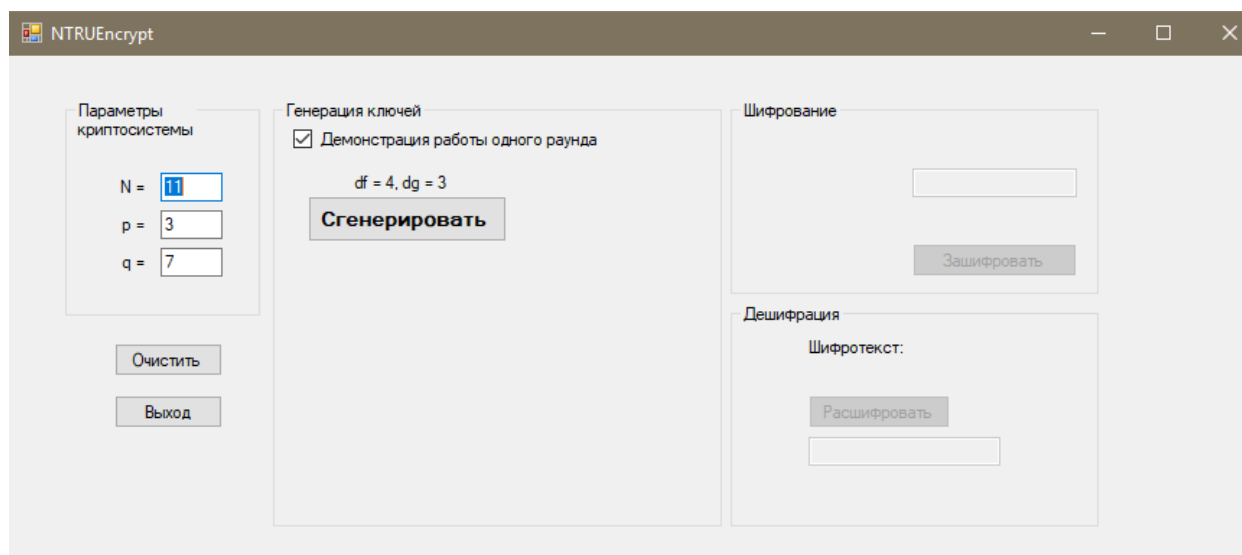


Рисунок 3.2 – Внешний вид программы NTRUEncrypt

NTRUEncrypt использует три постоянных параметра: N , p , q . Эти параметры вводятся пользователем на панели «Параметры системы», как показано на рисунке 3.3.

Числом N характеризуется размер выбираемых в качестве ключей многочленов и длина блока данных. Числа p и q не обязательно должны быть простыми, но их наибольший общий делитель НОД (p , q) должен равняться 1. Следует отметить, что параметр p служит для определения интервала, которому обязаны принадлежать все коэффициенты многочленов, используемых в криптографической системе. Параметры df и dg задают количество единиц в генерируемых программой полиномах f и g и принимают константные значения равные четырем и трем соответственно.

Параметры
криптосистемы

N = 11

p = 3

q = 7

Рисунок 3.3 – Ввод параметров

Прежде чем переходить к генерации ключей, нужно выбрать один из режимов работы программы. Для осуществления выбора служит специальный элемент на форме, который носит название CheckBox (флажок) с пометкой «Демонстрация работы одного раунда». Если флажок установлен, то будет активирован демонстрационный режим, в противном случае, будет активирована полноценная работа с файлом. Рассмотрим подробнее оба этих режима.

3.3.1 Демонстрационный режим работы программы NTRUEncrypt

Модуль генерации ключей. На вход ему подаются параметры, которые ввел пользователь на первом шаге алгоритма. Затем случайным образом формируются два полинома f и g , такие, чтобы количество коэффициентов равных единице и нулю было равно числу, которое в программе заранее определено, и чтобы степень полиномов соответствовала введенному параметру N .

Затем, используя теорему Евклида о нахождении наибольшего общего делителя для многочленов и ее обратный ход, вычисляем два обратных к f полинома по модулю p и q соответственно. Если по какой-то причине обратного полинома не существует, то обработчик исключений выдаст соответствующее предупреждение и тогда необходимо будет вернуться назад и сгенерировать новый полином f . Затем по формулам, приведенным в разделе 2.1.1, вычисляется открытый ключ h . рисунок 3.4.

Генерация ключей

☒ Демонстрация работы одного раунда

$df = 4, dg = 3$

Сгенерировать

$f = (x^1) + x^3 + x^4 + x^6 + (x^7) + x^8 + (x^9)$
 $g = x^0 + (x^1) + x^4 + (x^5) + (x^7) + x^9$
 $fp = -1\ 0\ 0\ 0\ 1\ -1\ 1\ -1\ 1\ 1$
 $fq = 2\ -2\ -3\ 1\ 1\ -3\ 2\ -3\ -1\ -2\ 2$
 $h = 1\ -1\ -2\ 3\ 2\ 1\ 3\ -1\ -3\ 0\ -3$

Открытый ключ: (h);
 Закрытый ключ: (f, fp).

Рисунок 3.4 – Генерация ключей

Пользователь вводит двоичный вид сообщения m на панели «Шифрование» в окне «Сообщение».

Чтобы провести операцию шифрования, программе необходимо выполнить предварительную подготовку, а именно – сгенерировать еще один полином, который называется «ослепляющим». Формируется он по такому же принципу, что и полиномы f, g .

Полученный до этого открытый ключ и полином g дают возможность зашифровать сообщение m , используя соответствующую формулу из раздела 2.1.2, затем результат проверяется на принадлежность кольцу усеченных многочленов, степени не превосходящей $N-1$ и выводится на экран, как видно из рисунка 3.5.

Шифрование

Битовое сообщение:

$g = (x^0) + x^2 + (x^3) + (x^5) + x^7 + x^{10}$

Зашифровать

Рисунок 3.5 - Шифрование

Дешифрация

Шифротекст:

Расшифровать

Рисунок 3.6 – Результат дешифровки

Следующий модуль реализует механизм дешифрации (раздел 2.1.3). Для этого выполняются последовательно действия с учетом кольца $Z[X]/(X^N - 1)$. Результат появится в окне, как показано на рисунке 3.6.

3.3.2 Режим работы с файлом программы NTRUEncrypt

Модуль генерации ключей данного режима работы программы абсолютно такой же, как и в демонстрационном режиме (раздел 3.3.1) поэтому нет смысла его рассматривать еще раз. Перейдем сразу к модулю шифрования сообщения из файла. Исходное сообщение содержится в файле F:\my_text.txt и имеет вид, как показано на рисунке 3.7.

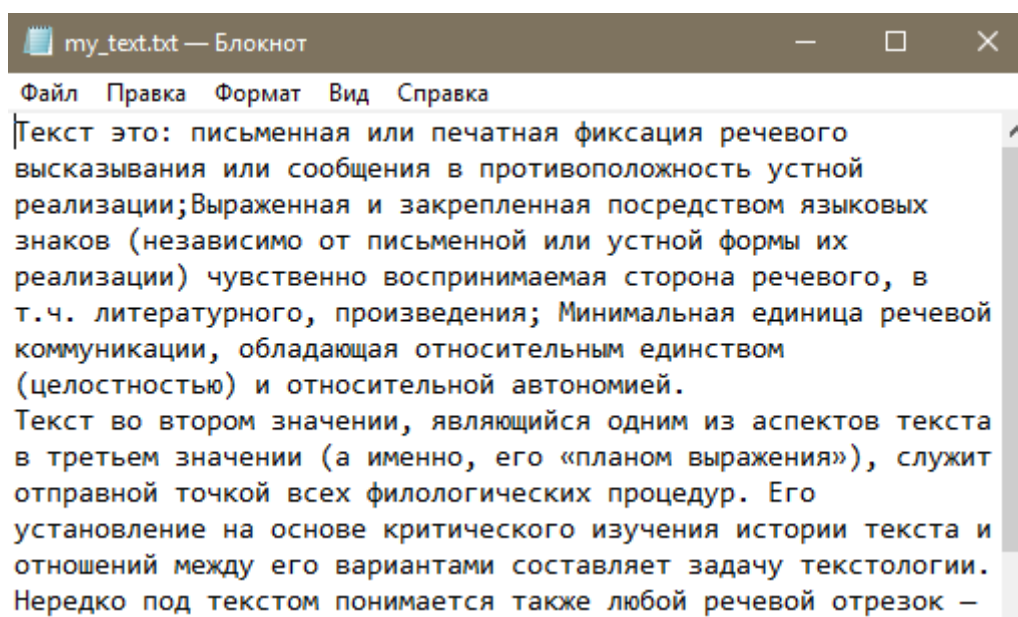


Рисунок 3.7 – Исходный текст

Стоит отметить, что исходное сообщение из файла разбивается на блоки по N бит, каждый из которых будет обрабатываться отдельно. После такого разбиения происходит генерация «ослепляющего» полинома g и шифрование каждого блока исходного сообщения, согласно формулам из раздела 2.1.2, рисунок 3.8.

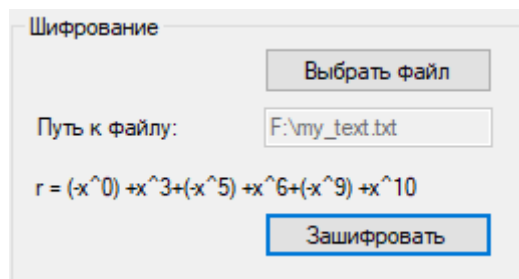


Рисунок 3.8 – Шифрование сообщения из файла

Затем каждый зашифрованный блок записывается последовательно в файл F:\shifr_text.txt и выглядит, как показано на рисунке 3.9.

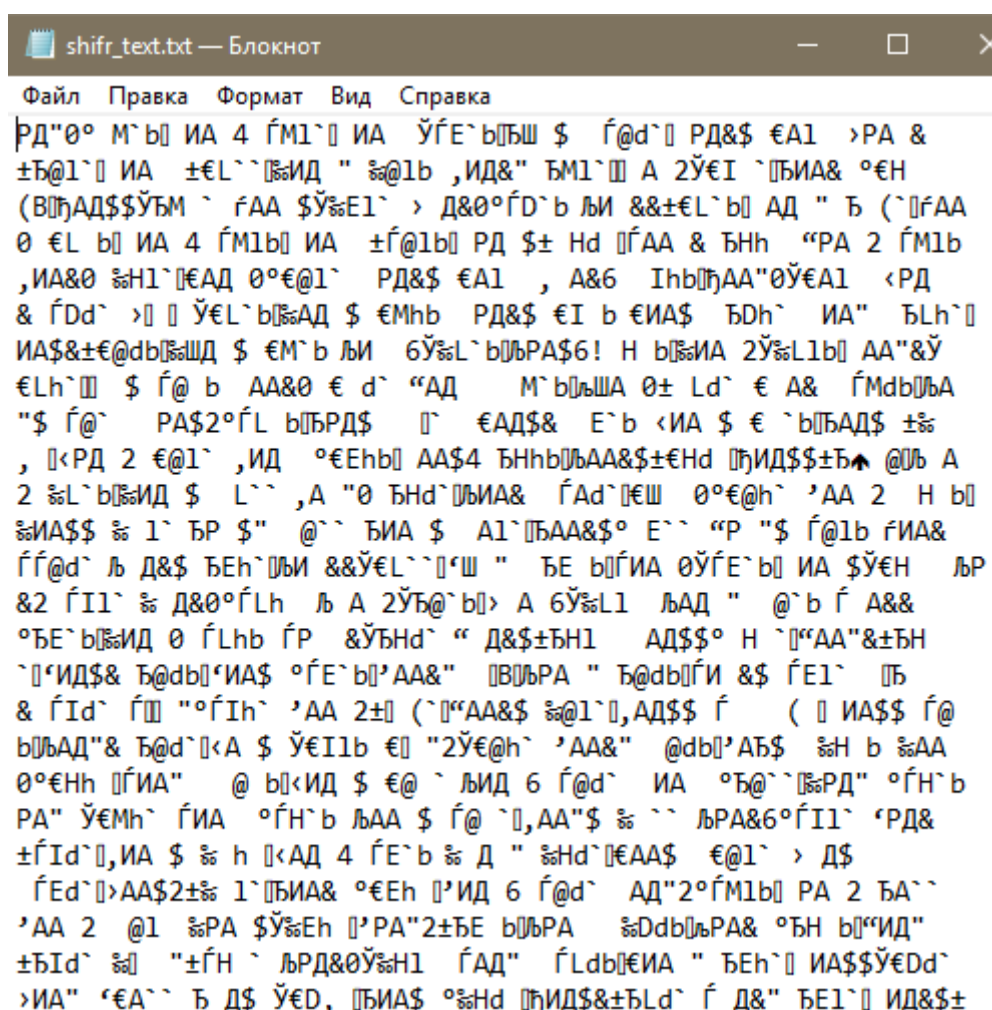


Рисунок 3.9 – Файл с шифротекстом

Для осуществления дешифрации необходимо файл с шифротекстом разбить на такие же блоки по N бит и применить к ним алгоритм, описанный в разделе 2.1.3, рисунок 3.10.

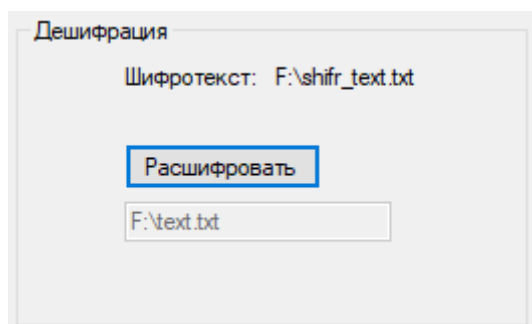


Рисунок 3.10 – Дешифрация в файл

Результат будет записан в файле F:\text.txt, содержимое которого показано на рисунке 3.11.

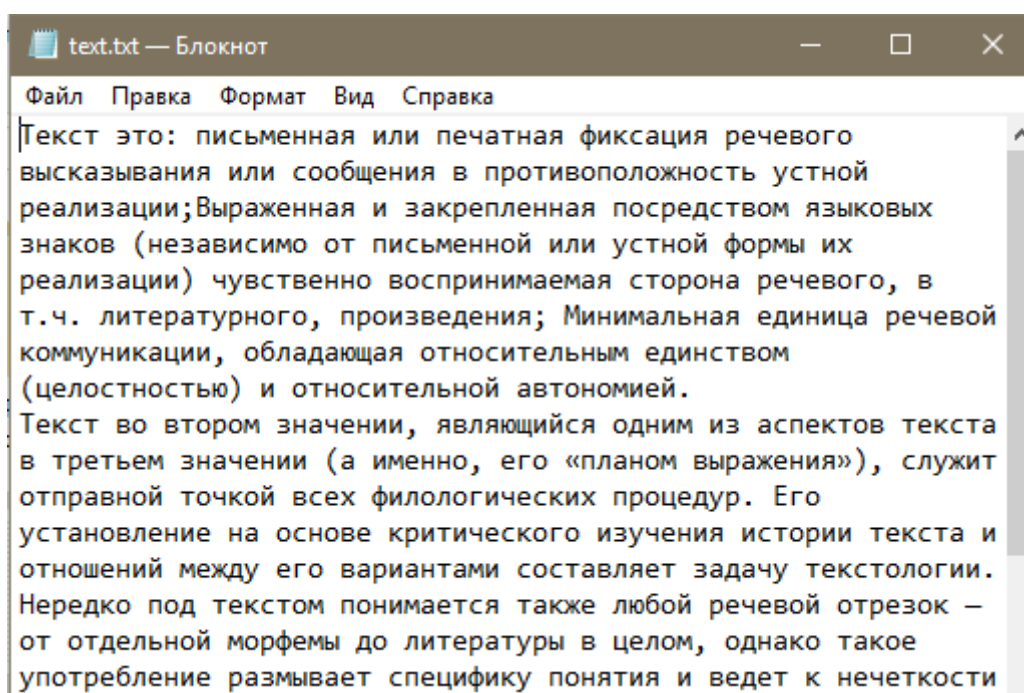


Рисунок 3.11 – Результат дешифрации в файле

3.4 Программная реализация модифицированного алгоритма

Описание того, в чем именно заключается модификация приведено в главе 2.2. Внешний вид модифицированной программы показан на рисунке 3.12.

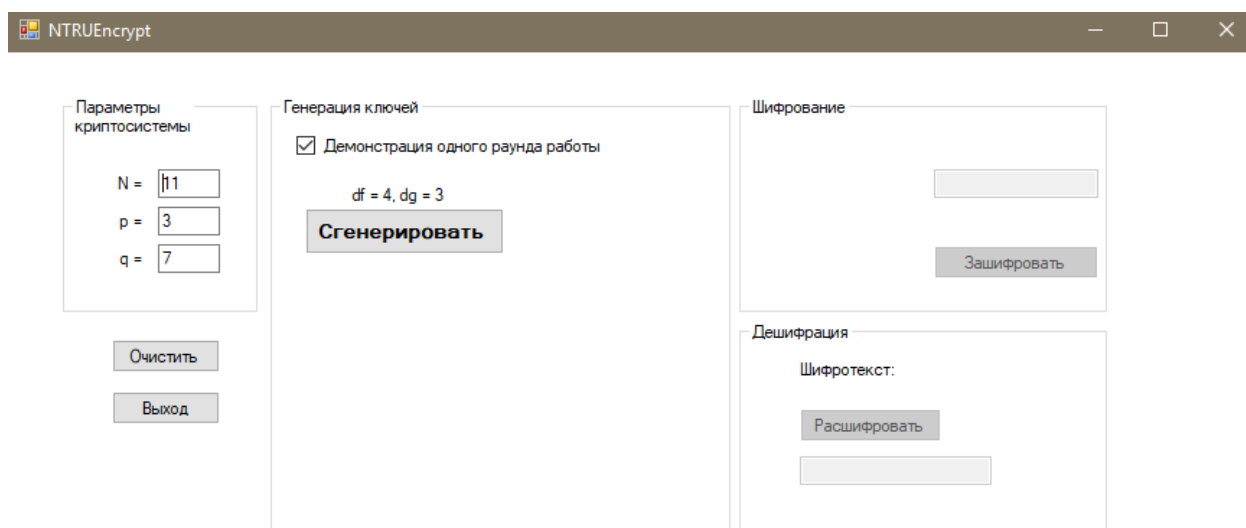


Рисунок 3.12 – Внешний вид модифицированного алгоритма

Первый модуль остается без изменений. Пользователь вводит параметры системы, как показано на рисунке 3.13:

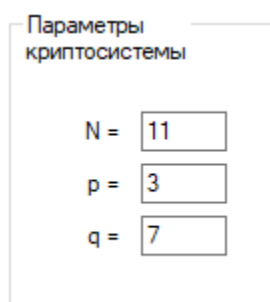


Рисунок 3.13 – Ввод параметров модифицированного алгоритма

Перед выполнением генерации ключей пользователь выбирает режим работы программы установкой или снятием флажка. Рассмотрим каждый из них более подробно.

3.4.1 Демонстрационный режим работы модифицированной программы

Модуль генерации ключей представляет собой наибольший интерес. Модификация касается алгоритма формирования полинома f и соответственно самого закрытого ключа. Как сказано в главе 2.2, вместо полинома f вычисляем полином $f = 1 + r \cdot F$, а значит пропадает необходимость вычисления полинома f_p и тогда вместо закрытого ключа (f, f_p) будем передавать сам полином (f) , все изменения показаны на рисунке 3.14:

Генерация ключей

☒ Демонстрация одного раунда работы

df = 4, dg = 3

Сгенерировать

$f = 1x^0 + (-3x^1) + 3x^2 + (-3x^4) + 3x^5 + (-3x^6) + 3x^9$
 $g = x^2 + (-x^3) + x^4 + x^5 + (-x^6) + (-x^7)$

fq = -3 -2 3 2 -1 3 3 0 -1 -2

h = 0 -1 1 0 1 2 3 0 0 3 -2

Открытый ключ: (h);
Закрытый ключ: (f).

Рисунок 3.14 – Генерация ключей модифицированного алгоритма

Блок шифрования остается без изменений, так как в данном модуле используется открытый ключ h и исходное битовое сообщение m , оба этих параметров модификация не затронула. Результат работы модуля шифрования показан на рисунке 3.15.

Что касается расшифрования или дешифрации, то сам алгоритм сократился, но внешне никаких изменений не произошло, рисунок 3.16.

Шифрование

Битовое сообщение:

$r = x^1 + x^2 + (-x^3) + (-x^5) + x^6 + (-x^8)$

Рисунок 3.15 – Шифрование модифицированного алгоритма

Дешифрация

Шифротекст: 1-3-1 2 1-1 1-2 2-3 2

Рисунок 3.16 – Дешифрация модифицированного алгоритма

3.4.2 Режим работы с файлом модифицированной программы

Внешний вид программы после выполнения всех модулей показан на рисунке 3.17.

NTRUEncrypt

Параметры криптосистемы

N =

p =

q =

Генерация ключей

☐ Демонстрация одного раунда работы

df = 4, dg = 3

$f = (-2x^0) + 3x^1 + 3x^2 + 3x^3 + 3x^5 + (-3x^9)$

$g = x^1 + x^2 + (-x^5) + (-x^7) + x^9 + (-x^{10})$

fq = 0-1 2 3 3-2-3 0 2-2

h = 3 2 0 1-1-2-2 2-2 1

Открытый ключ: (h);

Закрытый ключ: (f).

Шифрование

Путь к файлу:

$r = x^3 + (-x^5) + (-x^6) + x^8 + (-x^9) + x^{10}$

Дешифрация

Шифротекст:

Рисунок 3.17 – Режим работы с файлом модифицированного алгоритма

Подробнее останавливаться на каждом модуле не будем, так как все отличия модифицированной программы от NTRUEncrypt наглядно показаны на примере работы демонстрационного режима (раздел 3.4.1), а работа с файлом в точности повторяет алгоритм, заложенный в самой программе NTRUEncrypt

(раздел 3.3.2). Стоит лишь наглядно показать тот факт, что файл действительно расшифровывается, рисунок 3.18.

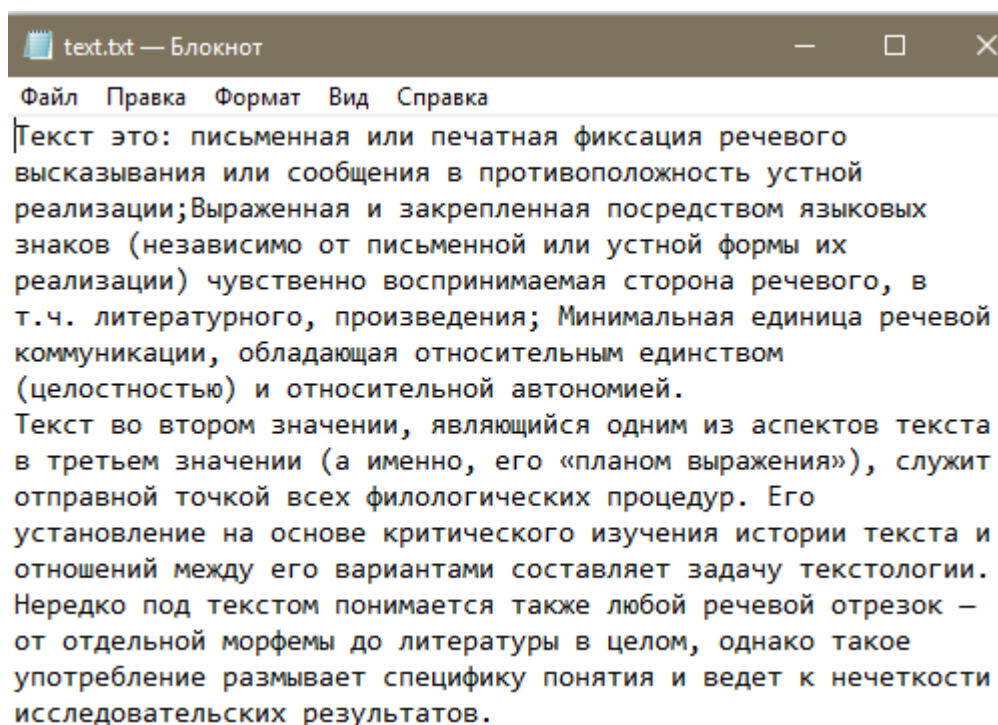


Рисунок 3.18 – Результат дешифрации в файле модифицированного алгоритма

Следующим шагом стоит оценить выигрыш модифицированной программы по сравнению со своим предком NTRUEncrypt и сравнить эти две разработки автора с классической криптосистемой – RSA.

3.5 Экспериментальное исследование

Целью экспериментального исследования является проведение сравнения разработанных криптосистем с их аналогом (в рамках данной работы в качестве альтернативной была выбрана криптосистема с открытым ключом RSA) по характеристикам производительности. А также провести экспериментальное сравнение реализованного автором алгоритма NTRUEncrypt со своей

модификацией (режим работы с файлом), обозначить все преимущества последней и привести экспериментальные доказательства.

3.5.1 Результаты экспериментального исследования

Для проведения эксперимента была использована сторонняя программная реализация криптографической системы RSA. В дополнение ко всему, в исходный код всех трех программ NTRUEncrypt, RSA, модификации NTRUEncrypt был внедрен элемент класса Stopwatch. Данный класс предоставляет набор методов и свойств, которые можно использовать для точного измерения затраченного времени. Таким образом, появилась возможность фиксирования результатов затраченного времени в зависимости от размера входного файла на всех трех основных этапах: создание ключей, шифрование сообщения, расшифрование.

Первый эксперимент проведен для сравнения модуля генерации ключей программы NTRUEncrypt и её модификации, данный эксперимент проводится отдельно по той причине, что скорость работы этого модуля не зависит от размера входного файла, результат представлен в таблице 1.

Таблица 1 – Результат времени работы модуля генерации ключей

№	Время работы NTRUEncrypt, сек	Время работы модификации NTRUEncrypt, сек
Генерация ключей		
1	0.0239676	0.0190014
2	0.0149743	0.0109994
3	0.0129915	0.0099936
4	0.0170503	0.0099772
5	0.0139885	0.0099773
Ср.знач:	0.01659444	0.01198978

Оценивая средние значения двух программ можно заметить очевидное превосходство модификации перед ее родителем. Модуль генерации ключей модифицированного алгоритма быстрее программы NTRUEncrypt на 27%. На рисунке 3.19 показан график данных по таблице 1.

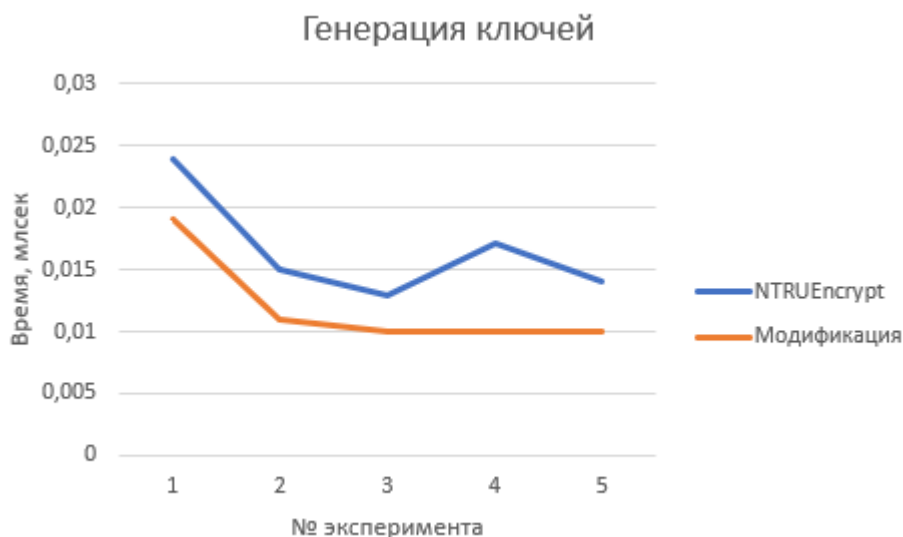


Рисунок 3.19 – График сравнения времени генерации ключей

Второй эксперимент проведем для сравнения времени работы модулей шифрования и дешифрации программы NTRUEncrypt и её модификации при разных размерах входных файлов (2 Кб, 6 Кб, 10 Кб, 20 Кб). Результаты эксперимента представлены в таблицах 2, 3, 4 и 5.

Таблица 2 – Результаты эксперимента для файла размером 2 Кб

№	Время работы NTRUEncrypt, сек	Время работы модификации NTRUEncrypt, сек
Шифрование файла		
1	0.0409773	0.0411910
2	0.0379942	0.0381968
3	0.0379654	0.0369637
4	0.0449722	0.0329992
5	0.0375791	0.0359822

Продолжение таблицы 2

Ср.знач:	0.0398976	0.0370665
Дешифрация в файл		
1	0.0549504	0.0379589
2	0.0559660	0.0409642
3	0.0498139	0.0389590
4	0.0920223	0.0399784
5	0.0529872	0.0589822
Ср.знач:	0.0611479	0.0433654

Таблица 3 – Результаты эксперимента для файла размером 6 Кб

№	Время работы NTRUEncrypt, сек	Время работы модификации NTRUEncrypt, сек
Шифрование файла		
1	0.0672165	0.0639591
2	0.0809674	0.0699759
3	0.0679745	0.0639819
4	0.0650731	0.0629639
5	0.0649622	0.0649868
Ср.знач:	0.0692387	0.0651635
Дешифрация в файл		
1	0.1129336	0.0696498
2	0.0978358	0.0704257
3	0.1083658	0.0819546
4	0.0979277	0.0739601
5	0.1139175	0.0689430
Ср.знач:	0.1061960	0.0729866

Таблица 4 – Результаты эксперимента для файла размером 10 Кб

№	Время работы NTRUEncrypt, сек	Время работы модификации NTRUEncrypt, сек
Шифрование файла		
1	0.0959449	0.1030077
2	0.0969475	0.0959272
3	0.1097316	0.0959417
4	0.102668	0.0919654
5	0.0879667	0.0969470
Ср.знач:	0.0981715	0.0967578
Дешифрация в файл		
1	0.1439006	0.1079292
2	0.1569131	0.1139166
3	0.1669071	0.1059413
4	0.1609238	0.1179315
5	0.1456907	0.1049438
Ср.знач:	0.1548670	0.1101324

Таблица 5 – Результаты эксперимента для файла размером 20 Кб

№	Время работы NTRUEncrypt, сек	Время работы модификации NTRUEncrypt, сек
Шифрование файла		
1	0.1999043	0.1978889
2	0.2008701	0.1018806
3	0.1984283	0.2008664
4	0.1998885	0.1948844
5	0.1698873	0.1969046
Ср.знач:	0.1737957	0.1784849

Продолжение таблицы 5

Дешифрация в файл		
1	0.3661662	0.2428463
2	0.3647892	0.2298673
3	0.3425901	0.2378478
4	0.3577952	0.2338651
5	0.3118367	0.2338519
Ср.знач:	0.3486354	0.2356556

Оценивая средние значения двух программ можно сделать вывод о том, что значения модуля шифрования приблизительно равны, а вот в рамках модуля дешифрации заметно значительное превосходство модифицированного алгоритма перед программой NTRUEncrypt. На рисунках 3.20 и 3.21 показан график шифрования и дешифрации данных по таблицам 2,3,4 и 5.

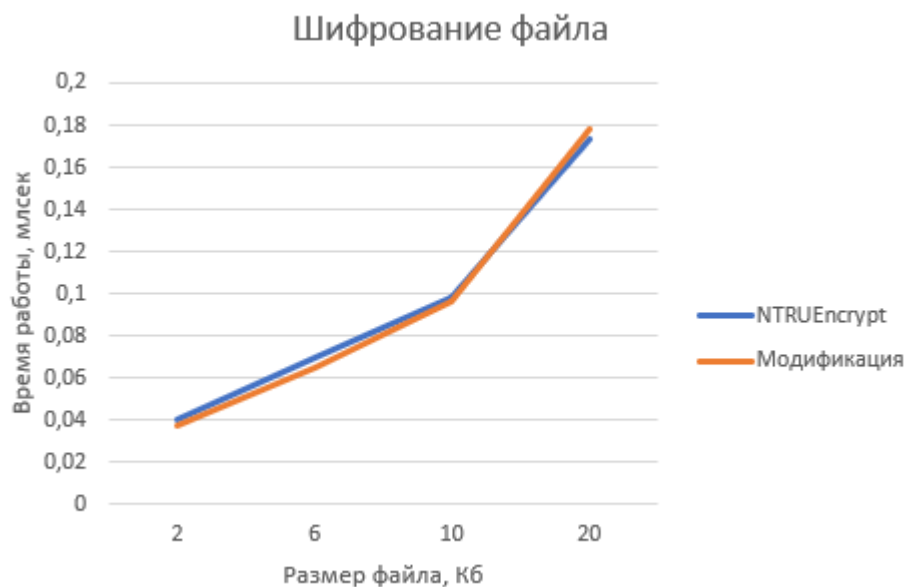


Рисунок 3.20 – График сравнения времени шифрования файла

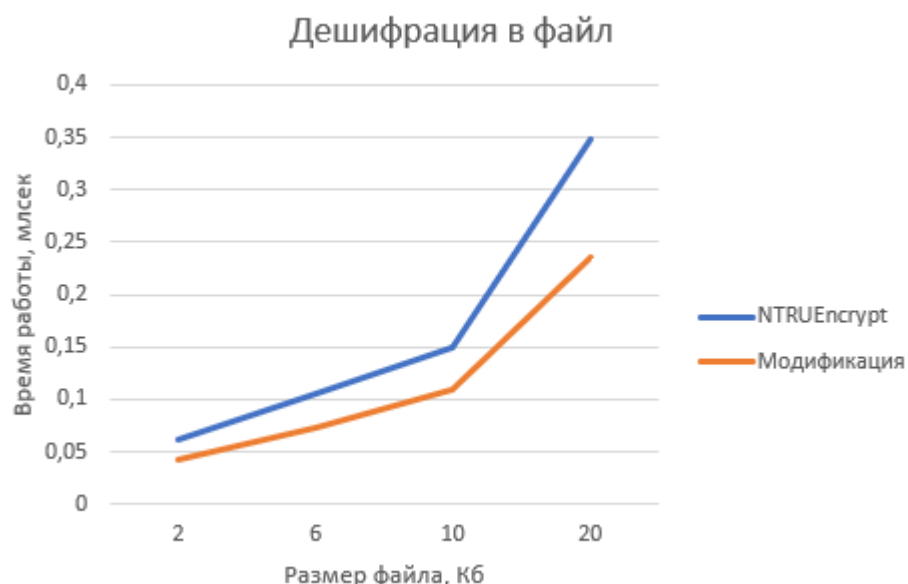


Рисунок 3.21 – График сравнения дешифрации в файл

Из графических данных можно сделать вывод, что с увеличением размера входного файла, увеличивается разница между графиками, т.е становится более заметным преимущество и рациональность использования модифицированного алгоритма. В процентном отношении, максимальное превосходство (в случае, когда исходный файл весит 20 Кб) составляет 32%.

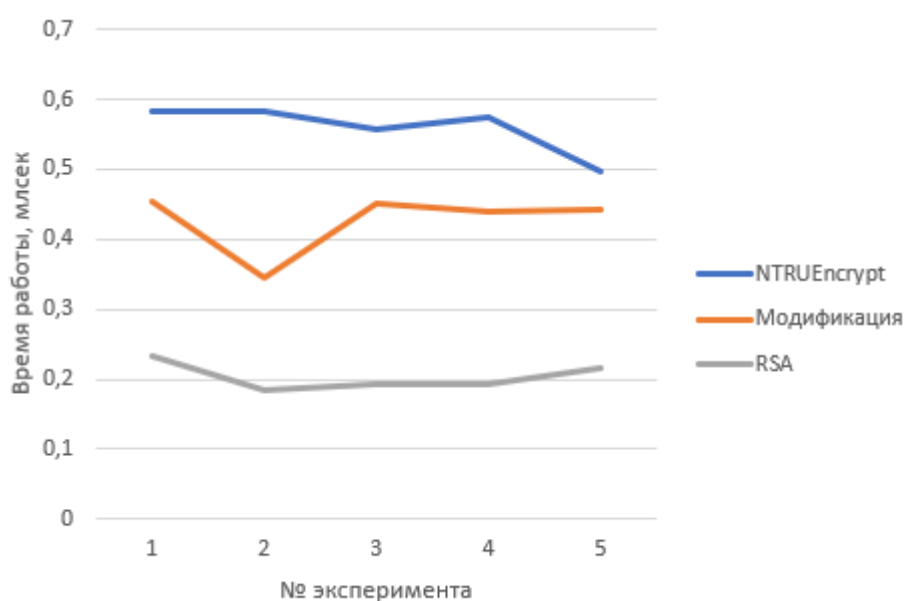


Рисунок 3.22 – График сравнения общего времени работы программ

Третий эксперимент заключается в сравнении разработанных автором программ с криптосистемой асимметричного шифрования RSA.

Отообразим графически, рисунок 3.22, результаты средних значений работы программ при одинаковых параметрах p и q , при одинаковом размере входного файла, для примера возьмем файл размером 20 Кб.

В результате видно преимущество модификации перед программой NTRUEncrypt, но, кроме этого, стоит заметить, что программа RSA существенно превосходит разработанные автором программы по скорости работы. Связано это с тем, что исходный код реализованных программ не оптимизирован и содержит намного больше строк кода – 900 против 200 строк кода в альтернативной программе RSA.

3.6 Выводы по третьей главе

1. Разработка программных продуктов. Описание выбранного языка программирования, общей структуры программ и приведена их обобщенная схема работы. Разработанные программные обеспечения представляют собой объектно-ориентированные приложения с простым и понятным интерфейсом. Состоят приложения из нескольких модулей и класса функций над полиномами, а также имеют два режима работы.

2. Приведены рисунки, демонстрирующие работу каждого режима работы программ и каждого модуля с дополнительными пояснениями автора.

3. Проведено три эксперимента. Получены результаты:

- модификация NTRUEncrypt быстрее программы NTRUEncrypt, в общем случае, на 20%;
- RSA быстрее программы NTRUEncrypt на 62% и быстрее её модификации на 52%.

ЗАКЛЮЧЕНИЕ

Для защиты от потери информации специалистами разработаны всевозможные механизмы и способы защиты, представляющие собой целый комплекс организационных, технических, программных средств с использованием всевозможных правовых норм, дающих возможность противодействовать всем источникам угроз безопасности информации на всех этапах работы с ней. В частности, для конфиденциальности и сохранности информации применяются криптографические и иные преобразования информации. В данной дипломной работе автором была реализована система, предоставляющая ресурс шифрования и дешифрования данных, криптографическая стойкость которой объясняется сложностью решения задачи теории решеток, заложенной в основу алгоритма NTRUEncrypt.

Автором было спроектировано и разработано программное обеспечение, реализующее работу криптосистемы NTRUEncrypt, а также программа, реализующая модификацию этой криптосистемы.

Экспериментально было доказано преимущество использования модификации алгоритма NTRUEncrypt. Разработанное приложение быстрее выполняет общую работу по генерации ключей, шифрованию и расшифрованию на 20%. Помимо этого, оптимизируется использование внутренней памяти за счет уменьшения размера исходного кода программы и размера секретного ключа. При этом сохраняется криптографическая стойкость и сложность использования квантовых алгоритмов при попытке взлома шифротекста.

Согласно статистическим данным криптологов из Лёвенского католического университета, алгоритм NTRUEncrypt превосходит криптосистему RSA в 4 раза по характеристикам скорости работы алгоритма. Но несмотря на статистику, сравнение разработанных автором программ со своим аналогом не подтвердило ожиданий. Работа алгоритма RSA оказалась

быстрее из-за недостаточной оптимизации исходного кода разработанных программ. Тема оптимизации является обширной и в перспективе должна быть поставлена задача достижение таких же высоких показателей скорости работы алгоритма, как в практических реализациях программы NTRUEncrypt, например, в финансовых приложениях.

Что касается стойкости NTRUEncrypt, то немаловажным будет отметить тот факт, что после создания квантовых компьютеров будут решены задачи быстрой факторизации и дискретного логарифмирования. Естественно, что в свете этого RSA, DSA и прочие им подобные алгоритмы становятся бесполезными. Но в случае с NTRUEncrypt дела обстоят иначе, ведь алгоритма, решающего задачу кратчайшего вектора решетки, не существует (хотя поиски активно ведутся с первой половины 90-х), а значит он вполне применим и в «постквантовую» эпоху.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Шаклеина, Т.А. «Мозговые центры» и их роль в формировании внешней политики США // Т.А. Шаклеина – Введение в прикладной анализ международных ситуаций. – Москва: ЗАО Издательство «Аспект Пресс», 2014. – с. 112.
2. Shor, P. Algorithms for quantum computation: discrete logarithms and factoring // P. Shor. – USA: 1994. – 124-134 с.
3. Алферов, А. П. Основы криптографии: учебное пособие / А.П. Алферов, А. Ю. Зубов, А. С. Кузьмин, А. В. Черемушкин // Москва: Гелиос АРВ – 2002.– 480 с. – с. 209-220
4. Лапониная, О. Р. Криптографические основы безопасности: учебное пособие / О. Р. Лапониная – Москва: Национальный Открытый Университет ИНТУИТ, 2016.– 242 с. – с. 118
5. Bakhtiari, M. Serious Security Weakness in RSA Cryptosystem / M. Bakhtiari, M. A. Maarof // IJCSI — 2012. — № 3. — P. 175–178. — ISSN 1694-0814; 1694-0784
6. Michele, M. Deep Blue, IBM's supercomputer, defeats chess champion Garry Kasparov in 1997 / M. Michele, K.C. Baker, S. Corky // New York Daily News. – 2015. – 10 мая.
URL: <http://www.nydailynews.com/news/world/kasparov-deep-blues-losingchess-champ-rooke-article-1.762264> (дата обращения: 28.10.2018).
7. Ишмухаметов, Ш. Т. Методы факторизации натуральных чисел: учебное пособие / Ш. Т. Ишмухаметов. — Казань: Казанский Университет, 2011. – 190 с. – 74 – 82 с.
8. Василенко, О. Н. Теоретико-числовые алгоритмы в криптографии: учебное пособие / О. Н. Василенко — Москва: МЦНМО, 2003.– 323 с. – 73-74 с.
9. Дирак, П.А.М. Принципы квантовой механики: 2-е издание / Дирак П.А.М. – Москва: Книга по Требованию, 2013. – 482 с. – 11-20 с.

10. Ajtai, M. The shortest vector problem in L2 is NP-hard for randomized reductions / M. Ajtai – USA: Electronic Colloquium for Computational Complexity, 1997.

11. Goldreich O., Goldwasser S., Halevi S. Collision-free hashing from lattice problems. Technical Report TR96-056. – Electronic Colloquium on Computation Complexity (ECCC). – 1996.

12. Kamel Bentahar, Dan Page, Markku-Juhani O. Saarinen, Joseph H. Silverman, Smart N. LASH. – NIST. Second Cryptographic Hash Workshop. – August 24, 2006.

13. V. Lyubashevsky, D. Micciancio, C. Peinkert, A. Rosen SWIFFT: a modest proposal for FFT hashing. In FSE 2008.

14. Arbitman Y., Dogon G., Lyubashevsky V., Micciancio D., Peikert C., Rosen F. SWIFFTX: A Proposal for the SHA-3 Standard. – In NIST. – 2008.

15. Goldreich O, Goldwasser, Halevi S. Public-key cryptosystems from lattice reduction problems. - In Advances in cryptology. Lecture Notes in Computer Science. – №1294. – 1997. – p. 112-131.

16. Gentry C., Peikert C., Vaikuntanathan V. Trapdoors for hard lattices and new cryptographic constructions. In Proc. 40th ACM Symp. on Theory of Computing (STOC). – 2008. – p. 197-206.

17. Micciancio D., Vadhan S. Statistical zero-knowledge proofs with efficient provers: lattice problems and more. In Advances in cryptology. Lecture Notes in Computer Science. – 2003. – p. 282-298.

18. Lyubashevsky V., Micciancio. Asymptotically efficient lattice-based digital signatures. Lecture in Computer Science. – 2008. – №4948. – p. 379–396

19. Peikert C., Vaikuntanathan V. Noninteractive statistical zeroknowledge proofs for lattice problems. In Advances in cryptology (CRYPTO), LNCS. – Springer. – №5157. – 2008. – p. 536-553.

20. Lyubashevsky V. Lattice-based identification schemes secure under active attacks. – In PKC. – №4939. – 2008. – p. 162-179.

21. Politi A., Jonathan C. F. V., O'Brien J.L. Shor's Quantum Factoring Algorithm on a Photonic Chip. – Science. – №325(5945). – 2009. – p. 1221.

22. Ladd T. D., Jelezko F., Laflamme R., Nakamura Y., Monroe C., O'Brien J. L. Quantum computers. – Nature. – №464. – 2010. – p. 45-53

23. Accredited Standards Committee X9

URL: <https://x9.org/> (дата обращения: 28.10.2018)

					10.05.01.480000.000 ПЗ	Лист
						71
Изм.	Лист	№ докум.	Подпись	Дата		

ПРИЛОЖЕНИЕ А

Техническое задание

«СОГЛАСОВАНО»

Руководитель проекта (работы)

_____ Л. В. Черкесова

«____» _____ 2018 г.

«УТВЕРЖДЕНО»

Зав. каф. «КБИС»

_____ А. И. Жуков

«____» _____ 2018 г.

А.1 Введение

А 1.1 Наименование программы

Наименование программы – «Программная реализация криптографической системы NTRUEncrypt».

А 1.2 Область применения

Областями применения данного программного средства могут быть сферы защиты информации.

А 1.3 Объект внедрения

Конечное программное средство предназначено для внедрения в организации ООО «Юзтех Интеграция».

А.2 Основания для разработки

Разработка ведется на основании документа «Учебный план для студентов ВУЗа», факультета «Кибербезопасность Информационных Систем»,

					10.05.01.480000.000 ПЗ	Лист
						72
Изм.	Лист	№ докум.	Подпись	Дата		

обучающихся по специальности 100501 «Компьютерная безопасность», в соответствии с которым студенты, заканчивающие ВУЗ, должны предоставить к защите работу, выполненную в ходе разработки дипломной работы.

А.3 Назначение разработки

А.3.1 Функциональное назначение

Функциональным назначением программного средства является реализация алгоритма NTRUEncrypt и его модификации, позволяющие выполнять операции шифрования и расшифрования входных файлов данных.

А.3.2 Эксплуатационное назначение

Эксплуатационное назначение – использование программного обеспечения в организациях, для которых необходимо применение средств защиты данных для их безопасной передачи по сети.

А.4 Требования к программному средству

А.4.1 Требования к функциональным характеристикам

Выполняемая функциональность разработанного программного средства:

- реализация режима демонстрации одного раунда – выполнение операций шифрования и расшифрования битового сообщения длины N бит;
- реализация режима работы с файлом;
- реализация функций генерации промежуточных полиномов – используются в модулях генерации ключей и шифрования исходного текста;

- реализация функций вычисления открытого и секретного ключей;
- реализация функции шифрования исходного сообщения;
- реализация функции дешифрации шифротекста.

Входные данные для функционирования программного средства:

- параметры криптосистемы;
- путь к файлу, содержащему исходное сообщение или битовое сообщение длины N бит.

А.4.2 Требования к надёжности

Надёжное функционирование программы должно быть обеспечено выполнением Заказчиком совокупности организационно-технических мероприятий, перечень которых приведен ниже:

- организацией бесперебойного питания технических средств;
- организацией правильного использования модулей;
- организацией правильного ввода параметров инициализирующих структур;
- регулярным выполнением требований ГОСТ 51188-98. Защита информации;
- испытания программных средств на наличие компьютерных вирусов.

А.4.3 Условия эксплуатации

Для функционирования программного продукта необходимо соблюдение всех требований и правил эксплуатации компьютерной техники. Дополнительные требований и ограничений не вводится.

					10.05.01.480000.000 ПЗ	Лист
						74
Изм.	Лист	№ докум.	Подпись	Дата		

Требования к персоналу, работающему с данным программным продуктом: общие знания вычислительной техники и знание предметной области.

А.4.4 Требования к составу и параметрам технических средств

Для работы программы требуется наличие IBM PC – совместимого персонального компьютера с тактовой частотой процессора не менее 1,5 Гц, объемом свободной оперативной памяти не менее 256 Мб, монитором и клавиатурой. Необходимое свободное место на жёстком диске – 200 Мб.

А.4.5 Требования к информационной и программной совместимости

Для корректного функционирования программного продукта необходима платформа .NET Framework 2.0 – 4.5.2. Совместима со всеми операционными системами Windows. Интегрированная среда разработки – Microsoft Visual Studio 12.0, язык программирования – C#.

А. 4.6 Требования к маркировке и упаковке

Требований к маркировке и упаковке не предъявлялось.

А.4.7 Требования к транспортированию и хранению

Условия транспортирования, места хранения, условия складирования и сроки хранения в различных условиях должны соответствовать требованиям, предъявляемым к носителям информации на которых будет содержаться данное программное изделие.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		75

Допустимы все способы транспортирования и хранения, не нарушающие целостность используемого носителя данных.

Программное средство может храниться на любом носителе информации, имеющее возможность подключения к ЭВМ.

А.5 Требования к программной документации

Программная документация должна включать следующие документы:

- документ «Техническое задание» (ГОСТ 19.201-78);
- документ «Руководство системного программиста» (ГОСТ 19.503-79);
- документ «Руководство программиста» (ГОСТ 19.504-79);
- документ «Руководство оператора» (ГОСТ 19.505-79).

А.6 Техничко-экономические показатели

Никаких технико-экономических требований к программе не предъявлено.

А.7 Стадии и этапы разработки

Этап 1. Системный анализ.

- изучение предметной области;
- определение области применения и целей использования разрабатываемого программного средства;
- поиск вариантов решения поставленных задач;
- подготовка технического задания.

Этап 2. Общесистемное проектирование.

- определение структуры программного комплекса;
- определение структуры алгоритмов и модулей.

					10.05.01.480000.000 ПЗ	Лист
Изм.	Лист	№ докум.	Подпись	Дата		76

Этап 3. Подготовка технологических средств.

- выбор языка программирования;
- выбор и подготовка инструментальных средств и средств отладки;
- разработка инструкций к применению методов.

Этап 4. Программная реализация, рабочий проект.

- разработка алгоритмической части;
- разработка текстов программных модулей;
- разработка документации;
- проектирование пользовательского интерфейса.

Этап 5. Отладка программного средства в статике.

- тестирование программных модулей;
- комплексирование модулей, поэтапное сведение в единый комплекс;
- локализация ошибок, корректировка исходных текстов,

информационных потоков, перекомпиляция.

А.7 Порядок контроля и приёмки

Порядок и контроль приемки определяются заведующим кафедрой «КБИС» и основаны на демонстрации знаний технологии и умении создавать программные средства для различных предметных областей. Главным требованием к приемке является наличие правильно работающего программного средства, иллюстрируемого тестовым примером и отчетом, представленным в печатном виде.

Разработчик технического задания:

Селёва Антонина Владимировна

«_____» _____ 2018г.

(подпись)

					10.05.01.480000.000 ПЗ	Лист
						77
Изм.	Лист	№ докум.	Подпись	Дата		

ПРИЛОЖЕНИЕ Б

Руководство системного программиста

Б.1 Общие сведения о программе

Областью применения данного программного комплекса являются сферы разработки приложений, использующих передачу конфиденциальных данных по сети. Разработанный программный комплекс предоставляет пользователям возможность шифрования файла, содержащего конфиденциальные данные с возможностью последующего его расшифрования, а также предусмотрен демонстрационный режим работы шифрования и расшифрования одного блока сообщения. На вход программе подается файл с данными с предварительным вводом параметров системы.

Программное обеспечение должно быть размещено на рабочей станции в состав которой входят:

- оперативное запоминающее устройства (ОЗУ), не менее 256 Мб;
- процессор с поддержкой x32 либо x64 набора инструкций;
- операционная система семейства Windows;
- платформа .NET Framework 2.0 – 4.5.2.

Б.2 Структура программы

Данное программное средство состоит из двух модулей:

- модуль формы, которая отвечает за взаимодействие пользователя с программой;
- модуль класса функций, реализующий операции над полиномами.

Б.3 Настройка программы

Программа предоставляет базовую функциональность сразу после своего запуска и не требует дополнительной настройки и конфигурирования. Кроме того, программа должны быть полностью укомплектована и должны быть соблюдены требования к аппаратной и программной среде, в которой выполняется программа.

Б.4 Проверка программного средства

Проверка работоспособности программы осуществляется при заданных входных параметрах, входного файла, либо входной последовательности N бит.

Б.5 Дополнительные возможности

На данном этапе, дополнительных функциональных возможностей не предусмотрено.

Б.6 Сообщения системному программисту.

Сообщений системному программисту не предусмотрено, кроме необходимых комментариев в самом исходном коде программного комплекса.

					10.05.01.480000.000 ПЗ	Лист
						79
Изм.	Лист	№ докум.	Подпись	Дата		

ПРИЛОЖЕНИЕ В

Руководство программиста

В.1 Общие сведения о программе

Областью применения данного программного комплекса являются сферы разработки приложений, использующих передачу конфиденциальных данных по сети. Разработанный программный комплекс предоставляет пользователям возможность шифрования файла, содержащего конфиденциальные данные с возможностью последующего его расшифрования, а также предусмотрен демонстрационный режим работы шифрования и расшифрования одного блока сообщения. На вход программе подается файл с данными с предварительным вводом параметров системы.

Программное обеспечение должно быть размещено на рабочей станции в состав которой входят:

- оперативное запоминающее устройства (ОЗУ), не менее 256 Мб;
- процессор с поддержкой x32 либо x64 набора инструкций;
- операционная система семейства Windows;
- платформа .NET Framework 2.0 – 4.5.2.

В.2 Характеристика программы

Программное средство реализует два режима работы: режим демонстрации шифрования и расшифрования блока битовых данных и режим работы с файлом, который производит шифрование и расшифрование файла данных в формате .txt. После запуска программы необходимо выбрать режим, установить параметры, сгенерировать ключи, а затем либо ввести блок битов в поле ввода, либо выбрать нужный файл.

					10.05.01.480000.000 ПЗ	Лист
						80
Изм.	Лист	№ докум.	Подпись	Дата		

В.3 Обращение к программе

Обращение к программе и работа с программой осуществляется путём запуска основного исполняемого модуля программы на машине под управлением одной из указанных в требованиях ОС, и, далее, осуществлением взаимодействия с программой через визуальный интерфейс.

В.4 Входные и выходные данные

В качестве входных данных демонстрационного режима одного раунда работы программы выступает набор битовых значений длины N , которые вводятся с клавиатуры в специальное поле ввода. Выходными данными, в случае работы этого режима, будут две последовательности такой же длины: первая – результат шифрования, вторая – результат дешифрации.

В качестве входных данных режима работы программы с файлом является файл с расширением .txt, содержащий исходные данные, которые необходимо зашифровать. Выходными данными, в случае работы этого режима, будут два файла: shifr_text.txt – результат шифрования, text.txt – результат дешифрации.

В.5 Сообщения

Сообщений системному программисту не предусмотрено, кроме необходимых комментариев в самом исходном коде программного комплекса.

ПРИЛОЖЕНИЕ Г

Руководство оператора

Г.1 Назначение программы

Областью применения данного программного комплекса являются сферы разработки приложений, использующих передачу конфиденциальных данных по сети. Разработанный программный комплекс предоставляет пользователям возможность шифрования файла, содержащего конфиденциальные данные с возможностью последующего его расшифрования, а также предусмотрен демонстрационный режим работы шифрования и расшифрования одного блока сообщения. На вход программе подается файл с данными с предварительным вводом параметров системы.

Г.2 Условия выполнения программы

Программное обеспечение должно быть размещено на рабочей станции в состав которой входят:

- оперативное запоминающее устройства (ОЗУ), не менее 256 Мб;
- процессор с поддержкой x32 либо x64 набора инструкций;
- операционная система семейства Windows;
- платформа .NET Framework 2.0 – 4.5.2.

Для успешного выполнения программы необходимо выполнение всех аппаратных и программных требований, а также необходимо использование корректных входных данных.

Г.3 Выполнение программы

Перед запуском программы необходимо более подробно ознакомиться с алгоритмом, заложенным в основу реализованного программного обеспечения.

После запуска приложения оператор указывает параметры алгоритма, если установленные по умолчанию его не устроили. Затем выбрать режим работы программы и сгенерировать ключи, нажав на соответствующую кнопку.

Демонстрационный режим работы программы. Ввести в поле ввода битовое сообщение длины N , затем зашифровать и расшифровать, нажав последовательно на соответствующие кнопки пользовательского интерфейса. Все результаты работы выводятся на экран.

Режим работы с файлом. Выбрать файл с исходным сообщением формата txt, которое необходимо зашифровать, а затем провести операции шифрования и расшифрования, нажав последовательно на соответствующие кнопки пользовательского интерфейса. Все результаты работы будут сохранены в соответствующих файлах и выведены на экран.

ПРИЛОЖЕНИЕ Д

Исходный код программного комплекса

Листинг Д.1 – Form1.cs

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.IO;
using System.Diagnostics;

namespace NTRUEncrypt
{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();
            openFileDialog1.Filter = "Text files (*.txt)|*.txt|All files (*.*)|*.*";
            toolTip1.SetToolTip(label1, "N - целое");
            toolTip1.SetToolTip(label2, " p, q : НОД(p, q) = 1");
            toolTip1.SetToolTip(label3, " p, q : НОД(p, q) = 1");
            toolTip1.SetToolTip(label4, "Полином f имеет df коэффициентов равных «1» и\n df-1 коэффициентов равных «-1», а остальные – «0».");
        }

        int df = 4, dg = 3, n, p, q ;
        double[] fq; double[] h_kolco; double[] shifr;
        double[] f; double[] shifr_kolco;
        double[] o_kolco; double[] hq;
        double[] g;

        void proverka_mod(double[] f, int modul)
        {
            for (int i = 0; i < f.Count(); i++)
            {
                if (f[i] > (modul - 1) / 2)
                    do
```



```

        {
            f[i] -= modul;
        }
        while (f[i] > (modul - 1) / 2);

    if (f[i] < (1 - modul) / 2)
        do
        {
            f[i] += modul;
        }
        while (f[i] < -(modul - 1) / 2);
    }

}

private void button1_Click(object sender, EventArgs e){
    label11.Visible = true;
    if (checkBox1.Checked)
    {
        textBox5.Enabled = true; textBox5.Text = "10011101001";
        label11.Text = "Битовое сообщение: ";
    }
    else
    {
        button6.Visible = true;
        button4.Text = "Выполнить";
        button5.Visible = false;
        label11.Text = "Путь к файлу: ";
    }

    ////////////задаем параметры
    int x;
    label5.Text = "f = "; label6.Text = "g = ";
    label8.Text = "fq = ";
    label10.Text = "h = ";
    label9.Text = "Открытый ключ: ( h );" + (char)13 + "Закрытый
ключ: ( f ).";
    try
    {
        n = Convert.ToInt32(textBox1.Text);
        p = Convert.ToInt32(textBox2.Text);
        q = Convert.ToInt32(textBox3.Text);

        panel1.Visible = true;
        List<double> newf = new List<double>();
        List<double> newg = new List<double>();
        Random rand = new Random();
        ////////////
        ////////////создаем полином f
        for (int i = 0; i < n; i++)

```

```

newf.Add(0);

for (int i = 0; i < df; i++)
{
    x = rand.Next(0, n);
    if (newf[x] != 0)
    {
        do
        {
            x = rand.Next(0, n);
            while (newf[x] != 0);
            newf[x] = 1;
        }
        else newf[x] = 1;
    }

for (int i = 0; i < df - 1; i++)
{
    x = rand.Next(0, n);
    if ((newf[x] != 0))
    {
        do
        {
            x = rand.Next(0, n);
            while ((newf[x] != 0));
            newf[x] = -1;
        }
        else newf[x] = -1;
    }

for (int i = n - 1; i > 0; i--)
    if (newf[i] == 0)
        newf.RemoveAt(i); //poslednie 0 del
    else break;

f = newf.ToArray();
for (int i = 0; i < f.Count(); i++)
    f[i] = f[i] * p;
f[0] = f[0] + 1;

for (int i = 0; i < f.Count() - 2; i++)
{
    if (f[i] < 0)
        label5.Text += " (" + f[i] + "x^" + i + ") +";
    if (f[i] > 0)
        label5.Text += f[i] + "x^" + i + " +";
}

    if (f[f.Count() - 1] < 0)
label5.Text += " (" + f[f.Count() - 1] + "x^" + f.Count() + ")";
    if (f[f.Count() - 1] > 0)
label5.Text += f[f.Count() - 1] + "x^" + f.Count();

```

```

//////////
//////////создаем полином g
for (int i = 0; i < n; i++)
    newg.Add(0);

for (int i = 0; i < dg; i++)
{
    x = rand.Next(0, 11);
    if (newg[x] != 0)
    {
        do
            x = rand.Next(0, 11);
        while (newg[x] != 0);
        newg[x] = 1;
    }
    else newg[x] = 1;
}

for (int i = 0; i < dg; i++)
{
    x = rand.Next(0, 11);
    if ((newg[x] != 0))
    {
        do
            x = rand.Next(0, 11);
        while ((newg[x] != 0));
        newg[x] = -1;
    }
    else newg[x] = -1;
}

for (int i = n - 1; i > 0; i--)
    if (newg[i] == 0)
        newg.RemoveAt(i); //последние 0 del
    else break;

g = newg.ToArray();

for (int i = 0; i < newg.Count() - 1; i++)
{
    if (newg[i] == 1)
        label6.Text += "x^" + i + '+';
    if (newg[i] == -1)
        label6.Text += "(-x^" + i + ") +";
}

if (newg[newg.Count() - 1] == 1)
    label6.Text += "x^" + (newg.Count() - 1);
if (newg[newg.Count() - 1] == -1)

```

```

label6.Text += "(-x^" + (newg.Count - 1) + ") + "
";
/////////////////////////////////
double[] chastnoe; double[] ostatok; // 1+x+x^2 - 1
1 1
List<double[]> Evklid = new List<double[]>();
double[] global = new double[n + 1]; // x^11 - 1;
global[n] = 1; global[0] = -1;
////////f
double[] c = { 0 };
////////// end f
//////////обратный по модулю q
int num1, num2;
Evklid.Clear();
Evklid.Add(global); //0
Evklid.Add(f); //1

Class1.Deconv(global, f, q, out chastnoe, out
ostatok);
proverka_mod(chastnoe, q); proverka_mod(ostatok,
q);
Evklid.Add(chastnoe); //2

newf.Clear();
newf = ostatok.ToList();
for (int i = ostatok.Count() - 1; i > 0; i--)
    if (newf[i] == 0)
        newf.RemoveAt(i); //poslednie 0 del
    else break;
ostatok = newf.ToArray();
Evklid.Add(ostatok); //3
num1 = 2; num2 = 4;
//

while (ostatok.Count() != 1)
{
    Class1.Deconv(Evklid[num1 - 1], Evklid[num2 -
1], q, out chastnoe, out ostatok);
    proverka_mod(chastnoe, q);
    newf.Clear();
    newf = ostatok.ToList();
    for (int i = ostatok.Count() - 1; i > 0; i--)
        if (newf[i] == 0)
            newf.RemoveAt(i); //poslednie 0 del
        else break;
    ostatok = newf.ToArray();
    Evklid.Add(chastnoe); Evklid.Add(ostatok);
    num1 += 2; num2 += 2;
}

```

```

//////////
double[] obratny = new double[] { };
List<double[]> step_evklid = new List<double[]>();
Class1.Raznost(c, Evklid[2], out obratny);
proverka_mod(obratny, q);
step_evklid.Add(obratny);
Class1.Proiz(Evklid[2], Evklid[4], out obratny);
obratny[0] += 1;
proverka_mod(obratny, q);
step_evklid.Add(obratny);

if (Evklid.Count() >= 7)
{
    for (int i = 7; i < Evklid.Count(); i += 2)
    {
        double[] cs;

Class1.Proiz(step_evklid[step_evklid.Count() - 1], Evklid[i - 1],
out cs);

Class1.Raznost(step_evklid[step_evklid.Count() - 2], cs, out
obratny);

        proverka_mod(obratny, q);
        step_evklid.Add(obratny);
    }
}

////////коэффициент добавили к обратному
if (Evklid[Evklid.Count() - 1][0] < 0)
    Evklid[Evklid.Count() - 1][0] = q +
Evklid[Evklid.Count() - 1][0];

double m = Evklid[Evklid.Count() - 1][0];
fq = new double[step_evklid[step_evklid.Count() -
1].Count()];
for (int i = 0; i < step_evklid[step_evklid.Count()
- 1].Count(); i++)
{
    step_evklid[step_evklid.Count() - 1][i] *=
(Math.Pow(m, q - 2) % q);
    proverka_mod(step_evklid[step_evklid.Count() -
1], q);
    fq[i] = step_evklid[step_evklid.Count() -
1][i];
}

proverka_mod(step_evklid[step_evklid.Count() - 1],
q);

////////// считываем полином h
c = new double[1] { p };

```

```

    obratny = new double[] { };
    Class1.Proiz(c, fq, out obratny);
    proverka_mod(obratny, q);
    double[] h; h_kolco = new double[n];
    int j1;
    Class1.Proiz(obratny, g, out h);
    // proverka_mod(h, q);
    for (int i = 0; i < h.Count(); i++)
    {
        if (i < n)
            h_kolco[i] = h[i];
        else
        {
            j1 = i;
            do
            {
                j1 -= n;
                while (j1 >= n);
                h_kolco[j1] += h[i];
            }
        }
    }
    proverka_mod(h_kolco, q);

    ////////////////////////////////////Obratny for h
    Evklid.Clear();
    Evklid.Add(global); //0
    Evklid.Add(h_kolco); //1
    Class1.Deconv(global, h_kolco, q, out chastnoe, out
ostatok);
    proverka_mod(chastnoe, q); proverka_mod(ostatok,
q);

    Evklid.Add(chastnoe); //2

    newf.Clear();
    newf = ostatok.ToList();
    for (int i = ostatok.Count() - 1; i > 0; i--)
        if (newf[i] == 0)
            newf.RemoveAt(i); //poslednie 0 del
        else break;
    ostatok = newf.ToArray();
    Evklid.Add(ostatok); //3
    num1 = 2; num2 = 4;
    //

    while (ostatok.Count() != 1)
    {
        Class1.Deconv(Evklid[num1 - 1], Evklid[num2 -
1], q, out chastnoe, out ostatok);
        proverka_mod(chastnoe, q);
    }
    proverka_mod(ostatok, q);
    newf.Clear();

```

```

        newf = ostatok.ToList();
        for (int i = ostatok.Count() - 1; i > 0; i--)
            if (newf[i] == 0)
                newf.RemoveAt(i); //последние 0 del
            else break;
        ostatok = newf.ToArray();
        Evklid.Add(chastnoe); Evklid.Add(ostatok);
        num1 += 2; num2 += 2;
    }
    ///////////////
    obratny = new double[] { };
    step_evklid = new List<double[]>();
    Class1.Raznost(c, Evklid[2], out obratny);
    proverka_mod(obotratny, q);
    step_evklid.Add(obotratny);
    Class1.Proiz(Evklid[2], Evklid[4], out obratny);
    obratny[0] += 1;
    proverka_mod(obotratny, q);
    step_evklid.Add(obotratny);

    if (Evklid.Count() >= 7)
    {
        for (int i = 7; i < Evklid.Count(); i += 2)
        {
            double[] cs;

Class1.Proiz(step_evklid[step_evklid.Count() - 1], Evklid[i - 1],
out cs);

Class1.Raznost(step_evklid[step_evklid.Count() - 2], cs, out
obotratny);

                proverka_mod(obotratny, q);
                step_evklid.Add(obotratny);
            }
        }

        //коэффициент добавили к обратному
        if (Evklid[Evklid.Count() - 1][0] < 0)
            Evklid[Evklid.Count() - 1][0] = q +
Evklid[Evklid.Count() - 1][0];
        m = Evklid[Evklid.Count() - 1][0];
        hq = new double[step_evklid[step_evklid.Count() -
1].Count()];
        for (int i = 0; i < step_evklid[step_evklid.Count()
- 1].Count(); i++)
        {
            step_evklid[step_evklid.Count() - 1][i] *=
(Math.Pow(m, q - 2) % q);
            proverka_mod(step_evklid[step_evklid.Count() -
1], q);

```

```

        hq[i] = step_evklid[step_evklid.Count() -
1][i];
    }

    proverka_mod(step_evklid[step_evklid.Count() - 1],
q);

    ////////////////////////////////// вывод результатов
    for (int i = 0; i < h_kolco.Count(); i++)
        label10.Text += " " + h_kolco[i];
    for (int i = 0; i < fq.Count(); i++)
        label8.Text += " " + fq[i];
    button4.Enabled = true;
    //TimeSpan result = DateTime.Now - start;
    //MessageBox.Show(result.ToString());
    }
    catch { MessageBox.Show("Введите натуральные
параметры"); }

}

private void button3_Click(object sender, EventArgs e)
{
    this.Close();
}

private void button4_Click(object sender, EventArgs e)
{
    DateTime start = DateTime.Now;
    label14.Text = ""; textBox6.Text = "";
    label13.Visible = true;

    double[] M;
    if (textBox5.Text != "")
    {
        ////////////////////////////////// генерация r
        int dr = 3, x;
        Random rand = new Random();
        double[] r;
        List<double> newr = new List<double>();

        for (int i = 0; i < n; i++)
            newr.Add(0);

        for (int i = 0; i < dr; i++)
        {
            x = rand.Next(0, 11);
            if (newr[x] != 0)
            {
                do

```



```

        x = rand.Next(0, 11);
        while (newr[x] != 0);
        newr[x] = 1;
    }
    else newr[x] = 1;
}

for (int i = 0; i < dr; i++)
{
    x = rand.Next(0, 11);
    if ((newr[x] != 0))
    {
        do
            x = rand.Next(0, 11);
        while ((newr[x] != 0));
        newr[x] = -1;
    }
    else newr[x] = -1;
}

for (int i = n - 1; i > 0; i--)
    if (newr[i] == 0)
        newr.RemoveAt(i); //последние 0 del
    else break;

r = newr.ToArray();
label13.Text = "r = ";
for (int i = 0; i < newr.Count() - 1; i++)
{
    if (newr[i] == 1)
        label13.Text += "x^" + i + '+';
    if (newr[i] == -1)
        label13.Text += "(-x^" + i + ") +";
}

if (newr[newr.Count() - 1] == 1)
    label13.Text += "x^" + (newr.Count() - 1);
if (newr[newr.Count() - 1] == -1)
    label13.Text += "(-x^" + (newr.Count - 1) + ")

";

////////// end r
////////// считывание M
if (checkBox1.Checked)
{
    M = new double[textBox5.Text.Length];
    for (int i = 0; i < textBox5.Text.Length; i++)
        if (textBox5.Text[i] != '2')
        {
            if (textBox5.Text[i] == '1')
                M[i] = 1;
        }
    }

```

```

else M[i] = 0;

    }
    else M[i] = -1;
////////// end M
////////// шифрование сообщения shifr
double[] c;
Class1.Proiz(r, h_kolco, out c);
proverka_mod(c, q);
Class1.Sum(c, M, out shifr);
proverka_mod(shifr, q);

shifr_kolco = new double[n];
kolco();
proverka_mod(shifr_kolco, q);

label14.Visible = true;
for (int i = 0; i < shifr_kolco.Count(); i++)
    label14.Text += " " + shifr_kolco[i];

////////// end shifr
double[] o; o_kolco = new double[n];
Class1.Proiz(f, M, out o);
int j;
for (int i = 0; i < o.Count(); i++)
{
    if (i < n)
        o_kolco[i] = o[i];
    else
    {
        j = i;
        do
            j -= n;
        while (j >= n);
        o_kolco[j] += o[i];
    }
}
proverka_mod(o_kolco, p);
button5.Enabled = true;
}
else // если файл
{
    FileNTRU();
}
}

//TimeSpan result = DateTime.Now - start;
//MessageBox.Show(result.ToString());
}

```

```

public void kolco()
{
    int j;
    for (int i = 0; i < shifr.Count(); i++)
        if (i < n)
            shifr_kolco[i] = shifr[i];
        else
        {
            j = i;
            do
                j -= n;
            while (j >= n);
            shifr_kolco[j] += shifr[i];
        }
}

private void button5_Click(object sender, EventArgs e)
{
    //дешифрация битового сообщения
    DateTime start = DateTime.Now;
    textBox6.Text = "";
    double[] a;
    double[] a_kolco = new double[n];
    if (checkBox1.Checked)
    {
        Class1.Proiz(f, shifr_kolco, out a);
        proverka_mod(a, q);

        int j;
        for (int i = 0; i < a.Count(); i++)
        {
            if (i < n)
                a_kolco[i] = a[i];
            else
            {
                j = i;
                do
                    j -= n;
                while (j >= n);
                a_kolco[j] += a[i];
            }
        }
        proverka_mod(a_kolco, q);
        a_kolco = o_kolco;
        proverka_mod(a_kolco, p);

        for (int i = 0; i < a_kolco.Count(); i++)
            if (a_kolco[i] != -1)
                textBox6.Text += " " + a_kolco[i];
    }
}

```

```

        else textBox6.Text += " 2";
    }
}

private void button2_Click(object sender, EventArgs e)
{
    //очистка
    fq = new double[] { }; h_kolco = new double[] { }; shifr
= new double[] { };
    f = new double[] { }; shifr_kolco = new double[] { };
    o_kolco = new double[] { }; hq = new double[] { };
    g = new double[] { };
    label5.Text = "f = "; label6.Text = "g = ";
    label8.Text = "fq = ";
    label10.Text = "h = ";
    label13.Text = ""; label14.Text = "";
    textBox6.Text = "";
}

private void button6_Click(object sender, EventArgs e)
{
    //открытие файла
    if (openFileDialog1.ShowDialog() ==
DialogResult.Cancel)
        return;
    string filename = openFileDialog1.FileName;
    textBox5.Text = filename;
}

public void FileNTRU()
{
    //обработка файла
    label14.Visible = true;
    FileStream file = new FileStream(textBox5.Text,
FileMode.Open, FileAccess.Read);

    byte[] buf = new byte[n];
    int rr = -1;
    byte b;
    double [] M = new double[n];
    double[] All_M = new double[n * 8];
    double[] All_shifr = new double[n * 8];
    double[] All_text = new double[n * 8];
    string path = @"F:\shifr_text.txt";
    label14.Text = path;
    string path2 = @"F:\text.txt";
    textBox6.Text = path;
    try
    {
        if ((File.Exists(path)) || (File.Exists(path2)))
        {

```

```

        File.Delete(path);
        File.Delete(path2);
    }
}
catch (Exception ex)
{
    MessageBox.Show(ex.ToString());
}

while (rr != 0)
{
    rr = file.Read(buf, 0, buf.Length);
    int k = 0;
    for (int j = 0; j < rr; j++)
    {
        b = buf[j];
        for (int i = 0; i < 8; i++)
        {
            All_M[i + k] = (b >> i) & 0x01;
            All_M[i + k] = All_M[i + k];
        }
        k += 8;
    }

    int d = 0;
    while (d < n * 8)
    {
        for (int i = d; i < n + d; i++)
        { M[i - d] = All_M[i];
          M[i - d] = M[i - d];

        }
        ////////// shifr
        double[] c;
        Class1.Proiz(f, h_kolco, out c);
        proverka_mod(c, q);

        Class1.Sum(c, M, out shifr);
        proverka_mod(shifr, q);

        shifr_kolco = new double[n];
        kolco();
        proverka_mod(shifr_kolco, q);

        for (int i = d; i < n + d; i++)
        { All_shifr[i] = shifr_kolco[i - d]; }

        ////////// end shifr
        double[] o; o_kolco = new double[n];
        Class1.Proiz(f, M, out o);
    }
}

```

```

int j;
for (int i = 0; i < o.Count(); i++)
{
    if (i < n)
        o_kolco[i] = o[i];
    else
    {
        j = i;
        do
            j -= n;
        while (j >= n);
        o_kolco[j] += o[i];
    }
}
/////rashifr
double[] a;
double[] a_kolco = new double[n];
Class1.Proiz(f, shifr_kolco, out a);
proverka_mod(a, q);
k = 0;
for (int i = 0; i < a.Count(); i++)
{
    if (i < n)
        a_kolco[i] = a[i];
    else
    {
        k = i;
        do
            k -= n;
        while (k >= n);
        a_kolco[k] += a[i];
    }
}
proverka_mod(a_kolco, q);
a_kolco = o_kolco;
proverka_mod(a_kolco, p);
for (int i = d; i < n + d; i++)
{
    All_text[i] = a_kolco[i - d];
    a_kolco[i - d] = a_kolco[i - d];
}
d += n;
}

byte[] info = new byte[1];
byte[] get = new byte[n];
//запись результатов в файл
using (FileStream fs = new FileStream(path,
FileMode.Append, FileAccess.Write))
{
    for (int j = 0; j < n; j++)

```

```

        {
            for (int q = j * 8, m = 1; q < j * 8 + 8; q++, m *= 2)
            {
                if (All_shifr[q] == 1) //шифротекст
                {
                    get[j] += (byte)m;
                }
                if (All_shifr[q] == 0)
                {
                    get[j] += (byte)0;
                }
            }
            get[j] = get[j];
            fs.Write(get, 0, get.Length);
            fs.Close();
            get = new byte[n];
            using (FileStream fs = new FileStream(path2,
            FileMode.Append, FileAccess.Write))
            {
                for (int j = 0; j < n; j++)
                {
                    for (int q = j * 8, m = 1; q < j * 8 + 8; q++, m *= 2)
                    {
                        if (All_text[q] == 1)
                        {
                            get[j] += (byte)m;
                        }
                        if (All_text[q] == 0)
                        {
                            get[j] += (byte)0;
                        }
                    }
                    get[j] = get[j];
                }
                fs.Write(get, 0, get.Length);
                fs.Close();
            }
            file.Close();
            Process.Start(path);
            Process.Start(path2);
            button5.Enabled = true;
        }
    }
}

```